企业级 RAG 智能问答系统全栈实施指南

2026 终极完整版：从原理到生产落地

技术栈： 基于 Spring Boot 3.4 + Spring AI 1.0 + Milvus 2.6 + Ollama 的完整技术手册

适用场景： 阿里云低配 ECS (2C4G) 极限部署 / 标准企业级 GPU 集群部署

文档版本： v2026.03.09 最终版

完整目录导航

部分	章节	主题
第一部分	第 1-7 章	基础认知与架构设计
第二部分	第 8-11 章	基础设施极速搭建
第三部分	第 12-14 章	核心组件部署与调优
第四部分	第 15-21 章	后端开发实战 (Spring AI)
第五部分	第 22-24 章	前端交互与体验优化
第六部分	第 25-28 章	低配环境极限生存指南
第七部分	第 29-36 章	生产部署与运维
附录	A-F	命令速查与配置模板

- 企业级 RAG 智能问答系统全栈实施指南
  - 2026 终极完整版：从原理到生产落地
- 完整目录导航
- 第一部分：基础认知与架构设计
  - 第 1 章 RAG 技术全景解析
    - 1.1 什么是 RAG？
      - 核心概念简介
    - 1.2 RAG 系统核心流转
      - 四大核心组件
    - 1.3 为什么需要向量数据库？
      - 传统 SQL 数据库的局限
      - 向量数据库的优势
    - 1.4 典型应用场景
  - 第 2 章 系统架构与运行逻辑
    - 2.1 完整架构设计
    - 2.2 技术底座
    - 2.3 一个完整请求的流转过程
  - 第 3 章 硬件选型与路线规划

        - 3.1 两条实施路线选择
        - 3.2 阿里云服务器选型建议（Path A）
        - 3.3 硬件与业务逻辑分工
        - 3.4 内存资产负债表（4GB 内存分配参考）
        - 3.5 为什么 2GB 内存必崩无疑？
            - 内存赤字账本
            - 连锁反应
    - 第 4 章 核心名词与概念详解
        - 4.1 向量 (Vector) 与 Embedding
            - Embedding 模型工作原理
            - 关键参数详解
        - 4.2 相似度度量算法
        - 4.3 HNSW 索引详解
            - 核心参数
            - 索引构建示例
        - 4.4 SSE (Server-Sent Events) 协议
            - 与 WebSocket 对比
            - SSE 数据格式
        - 4.5 Token 与文本切片
            - 切片策略详解
            - 推荐配置
        - 4.6 OOM (Out Of Memory) 详解
        - 4.7 Docker 网络模式详解
    - 第 5 章 Spring AI 版本演进说明
        - 5.1 Spring AI 1.0 GA 正式发布（2025 年 5 月 20 日）
        - 5.2 GA 版本核心改进
            - GA 版本主要改进
        - 5.3 M6 到 GA 版本迁移指南
    - 第 6 章 Embedding 模型选型指南（2026 版）
        - 6.1 MMTEB 评估基准最新排名（ICLR 2025）
        - 6.2 模型选型决策树
        - 6.3 维度与性能权衡
        - 6.4 Ollama 中的 Embedding 模型管理
    - 第 7 章 文档分块策略进阶
        - 7.1 传统分块 vs 新兴分块策略
        - 7.2 Late Chunking 实现原理
            - 传统流程
            - Late Chunking 流程
        - 7.3 Spring AI 中的分块配置
        - 7.4 分块策略性能对比测试
- 第二部分：基础设施极速搭建
    - 第 8 章 Docker 核心概念与最佳实践
        - 8.1 Docker 与 Docker Compose 的关系
        - 8.2 架构黄金准则：One Process Per Container
        - 8.3 Docker 常用命令速查
    - 第 9 章 阿里云环境极速搭建

        - 9.1 阿里云 Ubuntu 22.04 极速安装 Docker
        - 9.2 核心避坑：配置镜像加速器 (必做)
        - 9.3 Windows 虚拟化报错排查
        - 9.4 内存扩容 (Path B 低配机器必做)
        - 9.5 系统级"深层脱水"（释放物理资源）
            - 9.5.1 移除云厂商冗余监控 (回血 ~200MB)
            - 9.5.2 开启 Swap 虚拟内存与内核优化
    - 第 10 章 Milvus 向量数据库深度解析
        - 10.1 核心定位：为什么我们需要 Milvus？
        - 10.2 核心架构：云原生与存算分离
        - 10.3 关键特性 (2025 最新进展)
            - 10.3.1 混合搜索 (Hybrid Search)
            - 10.3.2 极速索引算法
            - 10.3.3 动态 Schema & JSON
            - 10.3.4 Milvus Lite
        - 10.4 主流向量数据库对比 (2025)
        - 10.5 30 秒上手 Milvus Lite
        - 10.6 总结建议
    - 第 11 章 Milvus 2.6 新特性详解
        - 11.1 Milvus 2.6 核心升级（2025 年 6 月发布）
            - 重大性能突破
        - 11.2 2.6 版本三大优化方向
        - 11.3 升级到 Milvus 2.6 的配置变更
        - 11.4 2.6 版本新增索引类型
- 第三部分：核心组件部署与调优
    - 第 12 章 Milvus 部署与配置优化
        - 12.1 下载与启动 Milvus
        - 12.2 精化版 docker-compose.yml（2 核 4G 环境优化）
        - 12.3 Milvus 优化的六大原理
            - 12.3.1 核心组件：资源边界限制 (`deploy.resources.limits`)
            - 12.3.2 依赖逻辑启动 (`depends_on` + `condition`)
            - 12.3.3 Milvus 内部参数微调 (`environment`)
            - 12.3.4 数据持久化 (`volumes`)
            - 12.3.5 安全与性能配置 (`security_opt`)
            - 12.3.6 4GB 内存服务器的"生存法则"
        - 12.4 应用新配置步骤
        - 12.5 解决端口冲突
    - 第 13 章 Ollama 模型管理与调优
        - 13.1 Ollama 安装与配置
            - 13.1.1 一键安装脚本
            - 13.1.2 常见报错处理
            - 13.1.3 远程访问配置
        - 13.2 拉取模型（按路线选择）
            - 🌟 Path A (标准配置)
            - 💻 Path B (低配配置)
        - 13.3 BGE 向量模型避坑

        - 13.4 强制限制推理线程 (关键优化)
        - 13.5 模型选型与预热
        - 13.6 阿里云/腾讯云安全组提醒
    - 第 14 章 可视化与监控工具
        - 14.1 Portainer 安装与管理
            - 14.1.1 安装命令
            - 14.1.2 无法访问/超时排查
        - 14.2 Attu 可视化工具
        - 14.3 内存监控
            - 健康指标
- 第四部分：后端开发实战 (Spring AI)
    - 第 15 章 项目初始化与依赖冲突解决
        - 15.1 终极兼容版 pom.xml
        - 15.2 替换 POM 后的关键操作
        - 15.3 项目结构推荐
        - 15.4 Java 服务 Dockerfile 示例
    - 第 16 章 核心配置与自动装配陷阱
        - 16.1 application.yml 配置文件
        - 16.2 Spring Boot 启动调优 (JVM 限制)
        - 16.3 Spring AI 核心配置调优 (1.0.0-M6 版)
            - 16.3.1 强制手动配置方案 (VectorStoreConfig)
            - 16.3.2 YAML 配置对齐
    - 第 17 章 文档解析与分批向量化
        - 17.1 文档解析与入库服务 (IngestionService.java)
        - 17.2 高效 Ingestion（分批入库逻辑）
        - 17.3 维度冲突处理
    - 第 18 章 RAG 检索与 SSE 流式响应
        - 18.1 RAG 流式问答服务 (RagService.java)
        - 18.2 控制层 (ChatController.java)
        - 18.3 性能优化：如何达到 2s 响应？
    - 第 19 章 完整代码汇总 (Controller/Service/Config)
        - 19.1 完整项目结构
        - 19.2 主启动类 (PolicyRagApplication.java)
        - 19.3 跨域配置 (CorsConfig.java)
        - 19.4 完整 VectorStoreConfig.java
        - 19.5 完整 logback-spring.xml
    - 第 20 章 RAG 检索优化 (混合检索/Rerank)
        - 20.1 混合检索（Hybrid Search）
            - 单一向量检索的局限：
            - 混合检索方案：
        - 20.2 Rerank 重排序优化
            - 为什么需要 Rerank？
            - 集成 Rerank 模型：
            - Rerank 模型推荐：
        - 20.3 查询重写（Query Rewriting）
        - 20.4 检索性能优化检查清单

- 第 21 章 RAG 系统评估体系
  - 21.1 核心评估指标
  - 21.2 自动化评估脚本
  - 21.3 评估数据集构建
- 第五部分：前端交互与体验优化
  - 第 22 章 Vue3 项目搭建
    - 22.1 项目创建
    - 22.2 项目结构
  - 第 23 章 SSE 流式通信原生实现
    - 23.1 流式响应解析 (原生 Fetch)
    - 23.2 专业流式解析逻辑（使用 fetch-event-source）
  - 第 24 章 UI/UX 优化与 Markdown 渲染
    - 24.1 完整 Vue3 组件示例
    - 24.2 UI/UX 优化建议
- 第六部分：低配环境极限生存指南 (重点)
  - 第 25 章 2 核 4G 服务器生存法则
    - 25.1 2G 环境下的表现与对策
    - 25.2 个人实验环境的"极限调优"（无 GPU 必看）
      - 25.2.1 模型降级：以小博大 (最有效)
      - 25.2.2 内存救命药：开启 Swap
      - 25.2.3 Milvus 瘦身
      - 25.2.4 JVM 参数调优
    - 25.3 实验配置总结表
  - 第 26 章 内存管理与 Swap 调优
    - 26.1 4GB 内存的黄金配置
      - 26.1.1 内存资产负债表 (合理分配 4GB)
    - 26.2 Ollama 服务管理 (防内存常驻)
  - 第 27 章 组件资源配额化实战
    - 27.1 最终内存动态监控
      - 健康指标
    - 27.2 最终健康度监控
    - 27.3 每日凌晨例行维护脚本
  - 第 28 章 系统级深度脱水
    - 28.1 严禁本地编译
    - 28.2 一个关键的排查技巧 (如果 Java 报错)
    - 28.3 I/O 等待检查
    - 28.4 内存隐形占用深度排查与清理
      - 28.4.1 彻底清理 Docker 冗余（释放隐形占用）
      - 28.4.2 清理系统级缓存与临时文件
      - 28.4.3 揪出"隐形"内存杀手
      - 28.4.4 极端情况：调整 Swappiness
      - 28.4.5 强制 Ollama 释放模型内存
      - 28.4.6 检查并关闭不常用的 Linux 服务
      - 28.4.7 针对 Spring Boot 的最终警告
      - 28.4.8 验证优化效果
    - 28.5 统启动顺序优化

        - 28.5.1 正确的启动顺序
        - 28.5.2 启动失败应急方案
    - 28.6 内存泄漏深度排查
        - 28.6.1 内存泄漏常见原因
        - 28.6.2 内存泄漏排查工具
        - 28.6.3 定期维护脚本（推荐每日凌晨执行）
        - 28.6.4 内存监控告警（可选）
- 第七部分：生产部署与运维
    - 第 29 章 容器化部署方案
        - 29.1 Docker Compose 编排模板
        - 29.2 部署关键要点
        - 29.3 实战部署三步走
    - 第 30 章 故障排查与常见问题
        - 30.1 Ollama 常见问题
        - 30.2 Milvus 常见问题
        - 30.3 Docker 常见错误
        - 30.4 Spring AI 常见问题
        - 30.5 综合排查流程
    - 第 31 章 自动化运维与监控
        - 31.1 运行健康指标 (Checklist)
        - 31.2 日志管理
        - 31.3 数据备份
    - 第 32 章 安全与性能最佳实践
        - 32.1 安全性建议
        - 32.2 性能最佳实践
        - 32.3 进阶建议
    - 第 33 章 关键遗漏点补充
        - 33.1 网络与安全组配置详解
        - 33.2 模型量化详解
        - 33.3 Prompt 工程优化
        - 33.4 文档上传最佳实践
        - 33.5 性能基准测试
        - 33.6 文档预处理最佳实践
        - 33.7 监控与告警配置
        - 33.8 灾难恢复计划
    - 第 34 章 企业级 RAG 安全与合规
        - 34.1 数据安全五层防护体系
        - 34.2 合规要求
        - 34.3 阿里云安全组完整配置
        - 34.4 数据出境合规性检查
        - 34.5 敏感信息过滤
    - 第 35 章 故障排查决策树 (增强版)
        - 35.1 三级风险管控体系
        - 35.2 红色警报：系统完全不可用
        - 35.3 黄色预警：性能下降
        - 35.4 常见错误代码速查

- 第 36 章 性能基准测试与优化目标
  - 36.1 性能测试工具
  - 36.2 性能目标基准（2026 年行业标准）
  - 36.3 性能优化优先级
  - 36.4 优化目标对照表
- 第 37 章 监控告警体系
  - 37.1 Prometheus + Grafana 监控配置
  - 37.2 关键监控指标
  - 37.3 告警通知配置
- 第 27.12 章 灾难恢复与备份策略
  - 27.12.1 数据备份方案
  - 27.12.2 备份策略建议
  - 27.12.3 灾难恢复流程

- 附录：命令速查与配置模板
  - 附录 A：完整配置文件模板
    - A.1 docker-compose.yml (完整版)
    - A.2 application.yml (完整版)
  - 附录 B：完整 Java 代码示例
    - B.1 PolicyRagApplication.java
    - B.2 VectorStoreConfig.java
    - B.3 CorsConfig.java
    - B.4 ChatController.java
    - B.5 RagService.java
    - B.6 IngestionService.java
  - 附录 C：完整前端代码示例
    - C.1 package.json
    - C.2 ChatBox.vue (完整组件)
  - 附录 D：故障排查决策树
  - 附录 E：2026 年技术趋势与展望
    - E.1 RAG 技术演进方向和路线
    - E.2 2026 年值得关注的技术和推荐关注项目
    - E.3 成本优化建议
  - 附录 F：完整检查清单（2026 版）
    - F.1 部署前检查清单
    - F.2 组件部署检查
    - F.3 功能验证检查
    - F.4 生产环境检查
    - F.5 开源代码仓
- 🎉 结语
  - 本书涵盖的技术栈：
  - 从入门到生产的完整路径：
  - 最后提醒：

第一部分：基础认知与架构设计

第 1 章 RAG 技术全景解析

1.1 什么是 RAG？

RAG（Retrieval-Augmented Generation，检索增强生成） 是一种将信息检索与文本生成相结合的人工智能技术架构。它通过从外部知识库检索相关信息，并将其作为上下文提供给大语言模型，从而生成更准确、更有依据的回答。

📌 核心概念简介

术语	英文全称	简单解释
RAG	Retrieval-Augmented Generation	先检索相关知识，再让 AI 生成答案
Embedding	嵌入向量	把文字变成数字向量，让计算机理解语义
Vector Store	向量存储	专门存储向量的数据库，能快速找到相似内容
LLM	Large Language Model	大语言模型，如 Qwen、GPT 等
Token	词元	AI 处理文本的基本单位，中文约 1.5 字符=1 token

1.2 RAG 系统核心流转

```
┌──────────────┐    ┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│  用户提问    │ → │  向量化检索  │ → │  上下文组装  │ → │  LLM 生成回答 │
│              │    │  (Milvus)    │    │  (Spring)    │    │  (Ollama)    │
└──────────────┘    └──────────────┘    └──────────────┘    └──────────────┘
```

四大核心组件

组件	角色	技术实现	详细说明
📝 翻译官	将非结构化文档转化为高维向量	Embedding 模型 (bge-m3/bge-small-zh)	负责把文字 "翻译 "成向量，让计算机能理解语义相似度
🗄️ 记忆库	负责海量向量的存储、索引及相似度检索	Milvus 向量数据库	存储所有向量，能在毫秒级找到最相似的内容
🧠 决策大脑	结合检索出的上下文背景生成精准回答	LLM 大模型 (Qwen/Ollama)	根据检索到的政策片段，生成准确的答案
🖥️ 展示层	流式文字输出，展示引用来源	Vue3 + SSE	实时显示答案，让用户不用等待完整生成

1.3 为什么需要向量数据库？

传统 SQL 数据库的局限

能力	SQL 数据库	说明

能力	SQL 数据库	说明
✅ 精确匹配	擅长	能准确查找 "等于"、"大于" 等条件
✅ 结构化数据	擅长	适合存储表格形式的规整数据
❌ 语义理解	无法	无法理解 "猫" 和 "小猫" 的语义关系
❌ 模糊搜索	困难	LIKE 查询效率低，无法理解语义相似度

向量数据库的优势

能力	向量数据库	说明
✅ 语义相似度	专精	能理解 "人工智能" 和 "AI" 是相似概念
✅ 模糊匹配	擅长	即使措辞不同，也能找到相关内容
✅ 海量数据	支持	专为千万级、亿级甚至万亿级向量数据集设计
✅ 毫秒检索	支持	使用 HNSW 等索引算法，实现毫秒级检索

1.4 典型应用场景

场景	描述	技术要点
🏛️ 企业知识库问答	构建政务/企业政策问答系统，用户提问→检索相关切片→注入 LLM 上下文→生成准确回答	需要准确的引用来源追溯
🖼️ 多模态搜索	在数亿商品或图片库中，实现毫秒级的视觉相似度检索	需要 CLIP 等多模态 Embedding 模型
🛍️ 个性化推荐	将用户行为和商品特征向量化，实时推荐感兴趣的内容	需要实时更新用户向量
🛡️ 异常检测/风控	通过对比行为轨迹的向量距离，快速识别欺诈或系统故障	需要设定相似度阈值告警

第 2 章 系统架构与运行逻辑

2.1 完整架构设计

本项目采用典型的 RAG（检索增强生成）架构，将政务文档（PDF/Word）转化为可检索的知识库，结合大模型进行精准回答。

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              展示层 (Presentation)                        │
│                    Vue3 前端 + SSE 流式输出 + 引用来源展示                 │
└──────────────────────────────────────────────────────────────────────────┘
                                      ↑↓
┌──────────────────────────────────────────────────────────────────────────┐
│                              生成层 (Generation)                          │
```

```
│              Ollama 运行的 Qwen-7B/Qwen2.5-1.5B 模型                    │
└──────────────────────────────────────────────────────────────────────┘
                                  ↑↓
┌──────────────────────────────────────────────────────────────────────┐
│                           检索层 (Retrieval)                          │
│         用户问题向量化 → Milvus 检索最相关政策片段 (Top K)              │
└──────────────────────────────────────────────────────────────────────┘
                                  ↑↓
┌──────────────────────────────────────────────────────────────────────┐
│                           数据层 (ETL)                                │
│    Spring Boot 读取文档 → bge-small-zh/bge-m3 模型向量化 → 存入 Milvus  │
└──────────────────────────────────────────────────────────────────────┘
```

2.2 技术底座

技术组件	版本	说明	选择理由
Spring Boot	3.4.3（稳定版）	后端框架	生态完善，与 Spring AI 深度集成
Spring AI	1.0.0-M6（里程碑版）	AI 集成框架	简化 AI 应用开发，统一 API 接口
Milvus	v2.4.0+	向量数据库	开源、高性能、支持海量数据
Ollama	最新版	本地大模型运行平台	本地部署、模型管理方便
Vue3	最新版	前端框架	响应式、组件化、生态丰富
JDK	21 LTS	Java 运行环境	长期支持版本，性能优化

2.3 一个完整请求的流转过程

1. **解析阶段：** Spring Boot 通过 Apache Tika 解析上传的 PDF/Word 政策文件

   ```
   └─> Tika 自动识别文件类型，提取纯文本内容
   ```

2. **向量化：** 调用 bge-small-zh/bge-m3 模型将文字转为向量

   ```
   └─> 每 500 token 切分为一个片段，重叠 100 token 保持语义连贯
   ```

3. **检索阶段：** 在 Milvus 中进行相似度搜索，提取最相关的政策原文

   ```
   └─> 使用余弦相似度，返回 Top 3 最相关片段
   ```

4. **Prompt 组装：** 将"用户问题"+"政策片段"组装成一段"限定范围"的提示词

   ```
   └─> 告知 LLM 仅基于提供的上下文回答，减少幻觉
   ```

5. **推理与流式响应：**
    - Ollama 使用选定模型生成答案
    - 后端通过 SSE (Server-Sent Events) 协议将字符流实时推送到 Vue3 前端
    - 用户在 500ms-2s 内看到首字和完整逻辑

第 3 章 硬件选型与路线规划

3.1 两条实施路线选择

> 💡 请根据你手头的服务器硬件资源，选择对应的实施路线

特性对比	🌟 **Path A：标准企业级方案**	💻 **Path B：低配/个人实验方案**
适用场景	生产环境、演示汇报、高性能要求	个人学习、阿里云低配 ECS、无显卡环境
硬件底线	至少 16GB 内存，建议配备 NVIDIA 显卡	2 核 4G / 4 核 8G，纯 CPU 运算
对话模型	qwen:7b (4-bit 量化，逻辑严密)	qwen2.5:1.5b (极致轻量，速度极快)
向量模型	bge-m3 (多语言优异，维度 1024)	bge-small-zh (极小巧，维度 512)
预期性能	首字延迟 < 1s，推理顺滑	首字延迟 < 2s，勉强流畅
保命手段	模型常驻内存、GPU 加速	开启 Swap 虚拟内存、模型降级、限制 JVM

3.2 阿里云服务器选型建议（Path A）

> ⚠ 要实现 2s 以内的响应延迟，硬件性能是决定性因素

组件	推荐配置	理由	预估成本
实例规格	阿里云 GPU 实例 (gn7i 或 gn6i)	必须配备 NVIDIA GPU，否则 7B 模型推理将极其缓慢（>10s）	约 8-15 元/小时
GPU	NVIDIA A10 (24GB) 或 T4 (16GB)	Qwen-7B 约占 5-8GB 显存，A10 可确保首字极速响应	包含在实例中
CPU/内存	8 核 32GB	Milvus 检索和 Spring Boot 文档解析非常消耗内存	包含在实例中
系统盘	100GB ESSD	确保模型加载和日志写入的高吞吐	约 50 元/月

> 🚨 **避坑提醒：** 普通 ECS 实例（无 GPU）只能以 CPU 模式运行 Ollama，体验极差，仅适合测试 1.5B 以下的超轻量模型。

3.3 硬件与业务逻辑分工

硬件	职责	说明	性能影响

硬件	职责	说明	性能影响
CPU	负责逻辑处理（Spring 业务、PDF 解析）和 AI 推理（Ollama 的数学计算）	核心数决定并发量	核心越多，并发处理能力越强
内存 (RAM)	系统的命门	Java (JVM)、Milvus 索引、Ollama 模型都需要驻留内存。内存不足会导致系统直接崩溃 (OOM)	内存不足是低配服务器最大瓶颈
GPU	AI 的加速器	没有它，AI 也能跑，但速度会变慢 10 倍以上	GPU 可使推理速度提升 10-50 倍
硬盘 (SSD)	决定向量数据库检索速度和模型加载速度	ESSD 优于普通云盘	SSD 可使数据加载速度提升 5-10 倍

3.4 内存资产负债表（4GB 内存分配参考）

> ⚠ 在 4G 内存中运行 Milvus + Ollama + Spring Boot，必须执行"资源精细化管理"，严防 OOM（内存溢出）导致系统假死

组件	内存分配上限	调优核心手段	监控命令
Milvus 数据库	1.0 GB	Docker deploy.resources.limits 硬限制 + 内部缓存限额	docker stats
Ollama (LLM+Embed)	2.2 GB	设置 KEEP_ALIVE 动态释放 + 限制并发线程	ollama ps
Spring Boot (JVM)	0.6 GB	参数 -Xmx600m -XX:MaxMetaspaceSize=256m	jstat -gc
OS 预留	0.2 GB	卸载冗余插件，维持基础响应	free -h

3.5 为什么 2GB 内存必崩无疑？

> 🕳 在 2GB 内存下，系统处于"小马拉大车"的超载状态

内存赤字账本

组件	内存需求	是否可优化
BGE-M3 (Embed)	~1.2GB	可降级为 bge-small-zh (~400MB)
Qwen-1.5B (LLM)	~1.0GB	可降级为 qwen2.5:0.5b (~400MB)
Milvus (DB)	~1.0GB	可限制为 512MB（影响性能）
Spring Boot	~0.5GB	可限制为 256MB（可能 OOM）
总计需求	**约 3.7GB（远超物理内存）**	**最低可压至 2.2GB**

连锁反应

```
物理内存溢出
    ↓
系统疯狂调用 Swap (虚拟内存)
    ↓
硬盘读写速度比内存慢数百倍
    ↓
CPU 长期处于 iowait 状态
    ↓
程序表现为连接超时 [request_sent] 或直接卡死
```

第 4 章 核心名词与概念详解

4.1 向量 (Vector) 与 Embedding

向量 是一组有序的数字序列，用于表示数据的数学特征。在 AI 中，文本、图片、音频等非结构化数据通过 Embedding 模型转换为向量。

Embedding 模型工作原理

```
原始文本："人工智能是未来技术"
    ↓ (通过 BGE-M3 模型处理)
向量：[0.123, -0.456, 0.789, ..., 0.321] (1024 维数字序列)
```

关键参数详解

参数	含义	典型值	影响说明
维度 (Dimension)	向量的长度，决定表达能力	512/768/1024/1536	维度越高，表达能力越强，但内存占用越大
量化 (Quantization)	压缩模型精度以减少内存占用	Q4_K_M/Q5_K_M/Q8_0	Q4 占用最小但精度损失约 3%，Q8 精度损失 <1%
上下文窗口 (Context)	模型一次能处理的最大 token 数	512/2048/4096	越大能处理的文本越长，但内存占用越高

4.2 相似度度量算法

算法	公式	适用场景	值域范围
余弦相似度 (Cosine)	$\cos(\theta) = A \cdot B / (\|A\| \|B\|)$	文本语义检索，最常用	[-1, 1]，越接近 1 越相似
欧式距离 (L2)	$\sqrt{\Sigma(a_i-b_i)^2}$	图像、物理空间距离	[0, ∞)，越小越相似
内积 (IP)	$\Sigma(a_i \times b_i)$	推荐系统、排序	无固定范围，越大越相似

> 💡 **Milvus 默认使用余弦相似度**，值域 [-1, 1]，越接近 1 表示越相似。一般阈值设为 0.6-0.8。

4.3 HNSW 索引详解

HNSW (Hierarchical Navigable Small World) 是一种基于图的近似最近邻搜索算法，是当前工业界综合性能最强的向量索引算法。

核心参数

参数	含义	推荐值	影响	内存影响
M	每个节点的最大连接数	16	越大检索越准，内存占用越高	M 每增加 8，内存增加约 10%
efConstruction	构建索引时的搜索深度	128	越大索引质量越好，构建越慢	仅影响构建时，不影响运行时
efSearch	检索时的搜索深度	64	越大检索越准，速度越慢	影响检索时内存和速度

索引构建示例

```yaml
spring:
  ai:
    vectorstore:
      milvus:
        index-type: HNSW
        index-params:
          M: 16
          efConstruction: 128
```

4.4 SSE (Server-Sent Events) 协议

SSE 是一种服务器向客户端推送实时数据的技术，基于 HTTP 长连接。

与 WebSocket 对比

特性	SSE	WebSocket	选择建议
通信方向	单向（服务器→客户端）	双向	RAG 问答只需单向，SSE 更简单
协议	HTTP/HTTPS	WebSocket (ws/wss)	SSE 无需额外协议支持
重连机制	自动	需手动实现	SSE 断线自动重连
适用场景	流式文本、新闻推送	聊天、游戏、实时协作	RAG 推荐 SSE

SSE 数据格式

```
data: 第
data: 一
data: 个
data: 字
data: [DONE]
```

4.5 Token 与文本切片

Token 是 NLP 中的基本处理单位，中文约 1.5 字符=1 token，英文约 0.75 单词=1 token。

切片策略详解

策略	描述	优点	缺点	适用场景
固定长度切片	按固定 token 数切割	实现简单，可预测	可能切断语义	通用场景
语义切片	按段落/句子边界切割	语义完整	实现复杂	文档结构清晰时
重叠切片	相邻切片保留部分重叠	防止语义截断	数据冗余	推荐默认使用

推荐配置

```
// 每片 500 token，重叠 100 token，保证语义连贯
TextSplitter splitter = new TokenTextSplitter(500, 100, 10, 10000, true);
```

4.6 OOM (Out Of Memory) 详解

OOM 是内存溢出错误，在 RAG 系统中常见原因：

原因	症状	解决方案	预防措施
JVM 堆内存不足	Java 进程被 kill	限制 -Xmx 参数	设置 -Xmx600m
容器内存无限制	容器占用全部物理内存	设置 deploy.resources.limits	Docker Compose 中限制
Swap 未开启	系统直接崩溃	创建 8G Swap 分区	部署前必做
模型并发加载	多个模型同时驻留内存	设置 OLLAMA_KEEP_ALIVE	设置 5 分钟自动释放

4.7 Docker 网络模式详解

模式	描述	适用场景	优缺点
bridge (默认)	容器通过虚拟网桥通信	大多数场景	✅ 隔离性好 ❌ 轻微性能损耗
host	容器直接使用宿主机网络	性能要求高	✅ 性能最优 ❌ 端口易冲突
none	无网络	安全隔离	✅ 最安全 ❌ 无法联网
container	共享其他容器网络	特殊架构	✅ 灵活 ❌ 配置复杂

> 💡 **RAG 系统推荐：** 使用默认 bridge 模式，通过服务名通信。

第 5 章 Spring AI 版本演进说明

5.1 Spring AI 1.0 GA 正式发布（2025 年 5 月 20 日）

> 📢 **重要更新：** 本书编写时使用的是 Spring AI 1.0.0-M6 里程碑版，但 Spring AI 1.0 GA 已于 2025 年 5 月 20 日正式发布。

版本	发布时间	状态	建议	主要变化
1.0.0-M1 ~ M8	2024-2025	里程碑版	仅用于学习测试	API 不稳定，可能有 breaking changes
1.0.0 GA	2025-05-20	正式生产版	生产环境推荐使用	API 稳定，生产就绪
1.1 GA	2025-11	正式版	新功能尝鲜	引入 Agents 框架
1.0.2	2026-01	最新稳定版	最佳选择	Bug 修复，性能优化

5.2 GA 版本核心改进

```
<!-- 推荐使用最新稳定版本 -->
<properties>
   <spring-ai.version>1.0.2</spring-ai.version>
</properties>

<dependencyManagement>
<dependencies>
   <dependency>
      <groupId>org.springframework.ai</groupId>
      <artifactId>spring-ai-bom</artifactId>
      <version>${spring-ai.version}</version>
      <type>pom</type>
      <scope>import</scope>
   </dependency>
</dependencies>
</dependencyManagement>
```

GA 版本主要改进

- **ChatClient API 稳定化：** 提供可移植且易于使用的标准接口
- **RAG 功能增强：** 改进检索增强生成的全流程支持
- **会话记忆机制：** 支持 ChatMemory 持久化到数据库
- **工具调用（Function Calling）：** 支持智能体（Agent）模式
- **评估框架（Bench）：** 内置 RAG 系统质量评估工具

5.3 M6 到 GA 版本迁移指南

变更项	M6 版本	GA 版本	迁移操作	影响程度
包路径	spring.ai.*	org.springframework.ai.*	更新 import 语句	🔴 高
ChatClient	ChatClient.builder()	ChatClient.create()	API 简化	🟡 中
VectorStore	自动配置不稳定	自动配置稳定	可移除手动配置	🟢 低
依赖仓库	需要 milestone 仓库	仅需 central 仓库	移除 milestone 配置	🟢 低

> ⚠️ **重要提醒：** 本书代码基于 M6 版本编写，若升级至 GA 版本，请参考官方迁移文档进行调整。

第 6 章 Embedding 模型选型指南（2026 版）

6.1 MMTEB 评估基准最新排名（ICLR 2025）

> 🎯 **震惊发现：** 参数量仅 560M 的模型击败了 7B 大模型！

排名	模型	参数量	维度	中文 MTEB 得分	内存占用	推荐指数	适用场景
🥇	bge-m3	567M	1024	64.8	~1.2GB	⭐⭐⭐⭐⭐	生产环境首选
🥈	bge-large-zh-v1.5	335M	1024	63.2	~800MB	⭐⭐⭐⭐	中文场景
🥉	text-embedding-3-large	-	3072	62.5	API 调用	⭐⭐⭐⭐	云端 API
4	bge-small-zh-v1.5	118M	512	61.8	~400MB	⭐⭐⭐⭐⭐ (低配)	低配服务器
5	m3e-base	220M	768	60.5	~600MB	⭐⭐⭐	平衡方案

> 📊 **数据来源：** MMTEB (Massive Multilingual Text Embedding Benchmark) 覆盖 250+ 语言、500+ 任务

6.2 模型选型决策树

```
是否需要中文支持？
    │
    ├─ 是 → 是否需要多语言？
    │        │
    │        ├─ 是 → bge-m3 (推荐)
    │        └─ 否 → bge-large-zh-v1.5
    │
    └─ 否 → 是否内存受限？
             │
             ├─ 是 → bge-small-zh-v1.5 (512 维)
             └─ 否 → text-embedding-3-large (3072 维)
```

6.3 维度与性能权衡

维度	检索精度	内存占用	检索速度	适用场景	推荐配置
512	85%	低	最快	低配服务器、快速原型	2C4G 环境
768	92%	中	快	平衡方案	4C8G 环境
1024	96%	高	中	生产环境推荐	8C16G+ 环境
3072	98%	极高	慢	高精度要求场景	GPU 环境

> ⚠ **重要提醒：** 更换 Embedding 模型后，必须删除并重建 Milvus Collection！维度不匹配会导致检索崩溃。

6.4 Ollama 中的 Embedding 模型管理

```
# 查看已安装的 Embedding 模型
ollama list | grep -E "embed|bge"

# 拉取推荐模型
ollama pull bge-m3

# 测试 Embedding 效果
curl http://localhost:11434/api/embeddings -d '{
  "model": "bge-m3",
  "prompt": "什么是 RAG 技术？"
}'

# 查看模型详细信息
ollama show bge-m3
```

第 7 章 文档分块策略进阶

7.1 传统分块 vs 新兴分块策略

策略	原理	优点	缺点	适用场景	实现难度
固定长度分块	按固定 token 数切割	实现简单，可预测	可能切断语义	通用场景	☆
重叠分块	相邻切片保留重叠	保持语义连贯	数据冗余	推荐默认使用	☆☆
语义分块	按段落/句子边界切割	语义完整	实现复杂	文档结构清晰	☆☆☆
Late Chunking	先 Embedding 再分块	检索精度提升 15%	计算成本高	高精度要求	☆☆☆☆
Max-Min 语义分块	动态相似度决策	自适应文档结构	需要调参	混合文档类型	☆☆☆☆☆

7.2 Late Chunking 实现原理

传统流程

```
文档 → 分块 → Embedding → 向量数据库
```

Late Chunking 流程

```
文档 → 句子 Embedding → 语义相似度计算 → 动态分块 → 向量数据库
```

> 💡 **优势：** 利用句子级向量计算语义相似度，确保分块边界在语义断裂处，检索精度提升 15%+。

7.3 Spring AI 中的分块配置

```
// 推荐配置：重叠分块（平衡性能与精度）
TokenTextSplitter splitter = new TokenTextSplitter(
                500,    // 每片 token 数
                100,    // 重叠 token 数（20% 重叠率）
                10,     // 最小片大小
                10000,  // 最大片大小
                true    // 保持段落完整
        );

// 低配服务器配置（减少内存占用）
TokenTextSplitter lowMemSplitter = new TokenTextSplitter(
        300,    // 减少每片大小
        50,     // 减少重叠
        10,
        5000,
        true
);
```

7.4 分块策略性能对比测试

分块大小	重叠率	检索精度	索引大小	检索延迟	推荐场景
200 token	10%	82%	小	20ms	低配服务器
500 token	20%	91%	中	35ms	生产环境推荐
800 token	25%	93%	大	50ms	高精度要求
500 token	0%	78%	最小	15ms	不推荐（语义断裂）

> 💡 **最佳实践：** 500 token + 20% 重叠率是精度与性能的最佳平衡点。

第二部分：基础设施极速搭建

第 8 章 Docker 核心概念与最佳实践

8.1 Docker 与 Docker Compose 的关系

特性	Docker (基础/单兵)	Docker Compose (编排/团队)
定义	用于创建和运行 **单个** 容器	用于定义和运行 **多容器** 应用程序
比喻	像是一块 **砖头**（单个容器）	像是 **建筑图纸**，规定砖头如何堆叠成房子
配置方式	命令行参数冗长（`docker run ...`）	YAML 配置文件（`docker-compose.yml`）
场景	测试单个服务、运行单一工具	运行包含 Java + Milvus + MySQL 的完整系统

8.2 架构黄金准则：One Process Per Container

> ✅ **结论：一个容器只部署一个微服务**

若强行在单一容器内运行多个服务（如 Nginx+MySQL+Java），会面临以下致命问题：

- **无法独立扩展：** 无法针对单个高负载服务进行动态扩容
- **部署耦合：** 修改一处代码需重启整个大容器，导致所有服务停机
- **进程管理困难 (PID 1 问题)：** 子服务崩溃可能无法触发 Docker 的自动重启机制
- **日志混乱：** 多服务日志交织，排错极其困难
- **违背微服务初衷：** 在物理层面强行耦合了逻辑上解耦的服务

8.3 Docker 常用命令速查

场景	命令	说明
启动服务	`docker compose up -d`	后台启动所有服务
停止并销毁	`docker compose down`	停止并移除容器、网络
实时日志	`docker logs -f <容器名>`	查看容器实时日志
进入容器	`docker exec -it <容器名> bash`	进入容器内部
清理空间	`docker system prune -a`	清理所有未使用资源
查看资源占用	`docker stats`	实时查看容器资源使用
列出模型 (Ollama)	`ollama list`	查看已下载模型
查看运行中 (Ollama)	`ollama ps`	查看正在运行的模型
删除模型 (Ollama)	`ollama rm <模型名>`	删除指定模型
拉取模型 (Ollama)	`ollama pull <模型名>`	下载新模型

第 9 章 阿里云环境极速搭建

9.1 阿里云 Ubuntu 22.04 极速安装 Docker

> CN **在阿里云国内环境下，请务必使用镜像源加速**

```
# 1. 一键安装 Docker 及 Compose 插件
sudo apt-get update
curl -fsSL https://get.docker.com | bash -s docker --mirror Aliyun

# 2. 设置开机自启
sudo systemctl enable docker
sudo systemctl start docker

# 3. 免 sudo 权限优化（执行后建议重新连接 SSH）
sudo usermod -aG docker $USER
newgrp docker
```

9.2 核心避坑：配置镜像加速器 (必做)

> ⚠ **国内拉取镜像极易报 `i/o timeout` 或 `TLS handshake timeout`，必须配置**

编辑 `/etc/docker/daemon.json`：

```
{
   "registry-mirrors": [
      "https://你的阿里云专属加速器地址",
      "https://docker.m.daocloud.io",
      "https://dockerproxy.com",
      "https://docker.nju.edu.cn",
      "https://docker.mirrors.sjtug.sjtu.edu.cn",
      "https://docker.mirrors.ustc.edu.cn"
   ]
}
```

生效命令：

```
sudo systemctl daemon-reload
sudo systemctl restart docker
```

> ⚠ **注意：** 若重启失败，请检查 JSON 格式（如逗号、引号是否匹配）。

9.3 Windows 虚拟化报错排查

报错	解决方案	优先级

报错	解决方案	优先级
WSL needs updating	运行 `wsl --update` 并 `wsl --shutdown`	🔴 高
Virtualization support not detected	BIOS 中开启 VT-x 或 SVM；Windows 功能中勾选 **虚拟机平台** 和 **适用于 Linux 的 Windows 子系统**	🔴 高

> 🏢 **阿里云 Windows ECS 警告：** 云服务器（虚拟机）默认不支持嵌套虚拟化。若在 Windows ECS 上跑 Docker 极度卡顿或报错，强烈建议重装为 Ubuntu 系统。

9.4 内存扩容 (Path B 低配机器必做)

> ⚠ **如果你的服务器内存不足 8G（如阿里云 2C4G），请务必先执行此步，否则后续启动 Milvus 或编译 Java 时系统会因 OOM（内存溢出）直接崩溃**

```
# 1. 创建 8G 虚拟内存文件并赋权
sudo fallocate -l 8G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile

# 2. 设置开机永久生效
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab

# 3. 验证是否成功 (查看 Swap 栏)
free -h
```

9.5 系统级"深层脱水"（释放物理资源）

9.5.1 移除云厂商冗余监控 (回血 ~200MB)

阿里云盾（AliYunDun）及其监控插件在内存告急时会产生明显的资源争抢和 CPU 采样开销。

```
# 运行官方卸载脚本
wget http://update.aegis.aliyun.com/download/uninstall.sh && chmod +x uninstall.sh
&& ./uninstall.sh
wget http://update.aegis.aliyun.com/download/quartz_uninstall.sh && chmod +x
quartz_uninstall.sh && ./quartz_uninstall.sh

# 强制清理残留进程
pkill aliyun-service && rm -rf /usr/local/aegis /usr/sbin/aliyun-service
```

9.5.2 开启 Swap 虚拟内存与内核优化

防止内存瞬间触底导致 OOM 杀掉核心进程。

```
# 1. 创建 8G 虚拟内存
sudo fallocate -l 8G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab

# 2. 调低 Swappiness (建议 10)
# 强制系统优先驻留物理内存，减少 I/O Wait (wa)
sudo sysctl vm.swappiness=10
echo 'vm.swappiness=10' | sudo tee -a /etc/sysctl.conf

# 3. 限制系统日志大小
sudo journalctl --vacuum-size=100M
```

第 10 章 Milvus 向量数据库深度解析

10.1 核心定位：为什么我们需要 Milvus？

> 💡 **一句话介绍：** Milvus 是一款由 Zilliz 发起、LF AI & Data 基金会顶级毕业项目的开源向量数据库。它是专为存储、索引和检索由深度学习模型生成的海量非结构化数据（Embedding）而设计的云原生基础设施。

在当前的 AI 浪潮（尤其是 RAG 检索增强生成）中，Milvus 是连接大模型与私有数据的关键桥梁。

10.2 核心架构：云原生与存算分离

Milvus 2.x 采用了先进的 **存算分离 (Disaggregated Storage and Compute)** 微服务架构，实现了弹性伸缩与高可用。

组件层级	名称	功能描述	依赖服务
接入层	Access Layer	系统的门户（Proxy），负责处理客户端请求与负载均衡	无
协调层	Coordinator	系统的 "大脑"，负责集群状态管理、任务分配与拓扑维护	etcd
执行层	Worker Nodes	Query Node：负责数据检索（计算密集型）Data Node：负责数据写入与持久化Index Node：负责构建高性能索引	无
存储层	Storage	依赖成熟的第三方存储：📦 对象存储 (S3/MinIO)📜 日志存储 (Kafka/Pulsar)🗂️ 元数据 (etcd)	MinIO, etcd, Kafka

> 💡 **架构优势：** 检索压力大？仅需扩容 Query Node；数据量剧增？仅需扩容存储。资源利用率最大化，成本更可控。

10.3 关键特性 (2025 最新进展)

10.3.1 混合搜索 (Hybrid Search)

从 Milvus 2.5 开始，多模态检索能力大幅增强：

- **稠密向量 (Dense)：** 处理语义理解（如 CLIP, BERT 模型生成的向量）
- **稀疏向量 (Sparse)：** 处理关键词匹配（SPLADE 等）
- **全文检索 (Full-text)：** 内置 BM25 算法，在一个库内同时实现"关键词 + 语义"的融合检索与重排 (Reranking)

10.3.2 极速索引算法

索引类型	特点	适用场景	内存需求
HNSW	工业界综合性能最强的内存图索引	中小规模数据，追求极致检索速度	高
DiskANN	磁盘索引，用有限的内存处理十亿级数据	大规模数据，降低硬件成本	低
GPU 加速	集成 NVIDIA CAGRA 算法，吞吐量提升数倍至数十倍	高并发生产环境	需 GPU

10.3.3 动态 Schema & JSON

支持存储复杂的元数据（Metadata）及 JSON 字段，支持在检索时进行标量过滤。

> 📌 **场景示例：** "查找语义上像'红色连衣裙'的图片，且价格在 100-500 元之间。"

10.3.4 Milvus Lite

开发者福音！无需 Docker 或 K8s，通过 Python 即可运行的轻量级版本，完美适配原型开发与 CI/CD 环境。

```
pip install pymilvus
```

10.4 主流向量数据库对比 (2025)

维度	Milvus 🚀	Pinecone	Weaviate	Qdrant
开源属性	✅ 完全开源	❌ 闭源 (SaaS)	✅ 开源	✅ 开源
核心优势	极致性能 & 扩展性	易用性 (Serverless)	混合搜索体验	Rust 编写、轻量级
部署方式	K8s, Docker, Cloud, Lite	仅限云端	Cloud, Docker	Cloud, Docker
适用场景	海量数据、生产级核心系统	快速验证、不想运维	语义搜索应用	推荐系统、RAG
亿级支持	⭐⭐⭐⭐⭐	⭐⭐⭐	⭐⭐⭐	⭐⭐⭐⭐

10.5 30 秒上手 Milvus Lite

```python
from pymilvus import MilvusClient

# 1. 初始化客户端（自动在本地创建 demo.db 文件）
client = MilvusClient("demo.db")

# 2. 创建集合（无需预定义 Schema，动态插入）
client.create_collection(
    collection_name="rag_knowledge_base",
    dimension=5  # 向量维度，例如 text-embedding-3-small 为 1536
)

# 3. 插入数据（包含向量和元数据）
data = [
    {"id": 1, "vector": [0.1, 0.2, 0.3, 0.4, 0.5], "text": "Milvus 架构解析"},
    {"id": 2, "vector": [0.9, 0.8, 0.7, 0.6, 0.5], "text": "RAG 系统设计"},
]
client.insert(collection_name="rag_knowledge_base", data=data)

# 4. 语义搜索
res = client.search(
    collection_name="rag_knowledge_base",
    data=[[0.1, 0.2, 0.3, 0.4, 0.5]],  # 查询向量
    limit=1,
    output_fields=["text"]  # 返回文本内容
)

print(f"检索结果：{res}")
```

10.6 总结建议

场景	推荐方案	理由
刚开始做 Demo	选 Milvus Lite，极速上手	无需 Docker，Python 直接运行
生产环境大规模应用	选 Milvus Cluster (K8s)，稳如磐石	高可用，弹性伸缩
不想运维基础设施	选 Zilliz Cloud，全托管省心	官方托管，免运维

> 💡 **Milvus 不仅仅是一个数据库，它是 AI 基础设施中不可或缺的长时记忆体。**

第 11 章 Milvus 2.6 新特性详解

11.1 Milvus 2.6 核心升级（2025 年 6 月发布）

重大性能突破

- 📉 **内存减少 72%：** 相同数据量下内存占用大幅降低
- 🚀 **检索速度提升 4 倍：** 比 Elasticsearch 快 4 倍

- 💰 **成本降低 60%：** 存算分离架构优化

11.2 2.6 版本三大优化方向

优化方向	具体改进	对本书项目的影响	优先级
降本增效	内存占用减少 72%，存储成本降低 60%	2 核 4G 服务器可运行更大数据量	🔴 高
搜索能力增强	全文检索功能强化，混合搜索优化	RAG 检索准确率提升 30%+	🟡 中
架构优化	底层存储引擎升级，支持更高层级并发	生产环境稳定性大幅提升	🔴 高

11.3 升级到 Milvus 2.6 的配置变更

```yaml
# docker-compose.yml 更新为 2.6 版本
services:
  standalone:
    container_name: milvus-standalone
    image: docker.m.daocloud.io/milvusdb/milvus:v2.6.0  # 升级版本号
    command: ["milvus", "run", "standalone"]
    environment:
      ETCD_ENDPOINTS: etcd:2379
      MINIO_ADDRESS: minio:9000
      # 2.6 版本新增优化参数
      KNOWHERE_GPU_CACHE_LIMIT: 512  # GPU 缓存限制
      QUERY_NODE_CACHE_MEM_RATE: 0.5  # 查询节点缓存内存比例
    volumes:
      - ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/milvus:/var/lib/milvus
    ports:
      - "19530:19530"
      - "9091:9091"
    deploy:
      resources:
        limits:
          memory: 1024M  # 2.6 版本可进一步降低
```

11.4 2.6 版本新增索引类型

索引类型	特点	适用场景	推荐配置	内存占用
HNSW	内存图索引，综合性能最强	中小规模数据（<1000 万）	M=16, efConstruction=128	高
DiskANN	磁盘索引，内存占用极低	大规模数据（>1 亿）	适合低配服务器	低
GPU_CAGRA	GPU 加速，吞吐量提升 10 倍	高并发生产环境	需要 NVIDIA GPU	中
TNT	2.6 新增，混合索引	混合检索场景	全文 + 向量联合查询	中

> 💡 **低配服务器建议：** Milvus 2.6 的 DiskANN 索引可在 4G 内存下处理千万级向量，强烈推荐 Path B 用户使用。

第三部分：核心组件部署与调优

第 12 章 Milvus 部署与配置优化

12.1 下载与启动 Milvus

> 💡 **推荐使用 Docker Compose 独立部署**

```
# 创建目录并进入
mkdir -p /usr/milvus && cd /usr/milvus

# 下载官方 Docker Compose 文件
wget https://github.com/milvus-io/milvus/releases/download/v2.4.0/milvus-standalone-docker-compose.yml -O docker-compose.yml

# 移除过时的 version 标签（新版 Compose 不需要）
sed -i '/version:/d' docker-compose.yml

# 启动服务
docker compose up -d

# 检查状态
docker compose ps
```

✅ 确保端口 `19530` 已开启

12.2 精化版 docker-compose.yml（2 核 4G 环境优化）

> ⚠️ **针对 2 核 4G 环境，在 `docker-compose.yml` 中做三件事：添加内存硬限制、优化启动顺序、针对小内存环境调优**

```
services:
   etcd:
      container_name: milvus-etcd
      image: quay.io/coreos/etcd:v3.5.5
      environment:
         - ETCD_AUTO_COMPACTION_MODE=revision
         - ETCD_AUTO_COMPACTION_RETENTION=1000
         - ETCD_QUOTA_BACKEND_BYTES=4294967296
         - ETCD_SNAPSHOT_COUNT=50000
      volumes:
         - ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/etcd:/etcd
      command: etcd -advertise-client-urls=http://127.0.0.1:2379 -listen-client-urls http://0.0.0.0:2379 --data-dir /etcd
      healthcheck:
```

```yaml
      test: ["CMD", "etcdctl", "endpoint", "health"]
      interval: 30s
      timeout: 20s
      retries: 3
    # --- 内存限制 ---
    deploy:
      resources:
        limits:
          memory: 256M  # etcd 占用较小，限制在 256M 足够

  minio:
    container_name: milvus-minio
    image: docker.m.daocloud.io/minio/minio:RELEASE.2023-03-20T20-16-18Z
    environment:
      MINIO_ACCESS_KEY: minioadmin
      MINIO_SECRET_KEY: minioadmin
    ports:
      - "9001:9001"
      - "9000:9000"
    volumes:
      - ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/minio:/minio_data
    command: minio server /minio_data --console-address ":9001"
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]
      interval: 30s
      timeout: 20s
      retries: 3
    # --- 内存限制 ---
    deploy:
      resources:
        limits:
          memory: 512M  # Minio 主要是存储，512M 足够日常使用

  standalone:
    container_name: milvus-standalone
    image: docker.m.daocloud.io/milvusdb/milvus:v2.4.0
    command: ["milvus", "run", "standalone"]
    security_opt:
      - seccomp:unconfined
    environment:
      ETCD_ENDPOINTS: etcd:2379
      MINIO_ADDRESS: minio:9000
      # --- 针对 4G 内存的性能调优参数 ---
      # 限制 Milvus 内部索引占用，强制让它在达到限制时进行内存回收
      CACHE_SIZE: 512M
    volumes:
      - ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/milvus:/var/lib/milvus
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:9091/healthz"]
      interval: 30s
      start_period: 90s
      timeout: 20s
      retries: 3
    ports:
```

```
            - "19530:19530"
            - "9091:9091"
        # --- 强制依赖健康状态 ---
        depends_on:
            etcd:
                condition: service_healthy
            minio:
                condition: service_healthy
        # --- 核心内存限制 ---
        deploy:
            resources:
                limits:
                    memory: 1024M  # 严控 Milvus 主程序占用在 1GB 以内

    networks:
        default:
            name: milvus
```

12.3 Milvus 优化的六大原理

12.3.1 核心组件：资源边界限制 (`deploy.resources.limits`)

> ⚠ **这是本次优化最关键的部分。** 在 Docker 中，如果不加限制，容器会尽可能占用宿主机的全部内存

组件	限制	解释	可调整范围
Etcd	256MB	存储 Milvus 的元数据（类似目录索引）。元数据通常很小，256MB 非常宽裕	128M-512M
Minio	512MB	存储实际的向量数据文件和日志。小规模 RAG 项目中压力不大	256M-1G
Milvus Standalone	1024MB	向量数据库的核心引擎，负责向量计算和索引。必须画出 "红线"	512M-2G

12.3.2 依赖逻辑启动 (`depends_on` + `condition`)

旧版逻辑： 启动 Etcd → 启动 Minio → 启动 Milvus

> ⚠ **风险：** Etcd 可能还没初始化完，Milvus 就尝试连接，导致 Milvus 启动失败并反复重启，瞬间推高 CPU 占用。

优化版逻辑：

- Milvus 会等待 Etcd 变成 `healthy`（健康）状态
- Milvus 会等待 Minio 变成 `healthy`（健康）状态

> ✅ **好处：** 避免了启动时的 CPU 瞬间峰值，确保系统组件一个接一个稳步运行，这对于 2 核的弱 CPU 环境极其重要。

12.3.3 Milvus 内部参数微调 (`environment`)

`CACHE_SIZE: 512M`：

- **原理：** Milvus 为了速度，默认会占用大量内存做缓存（Cache）来存储索引
- **设置意义：** 手动限制内部缓存为 512MB。这样加上程序运行所需的内存，总占用会很稳地控制在 1GB 左右

12.3.4 数据持久化 (`volumes`)

```
volumes:
  - ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/milvus:/var/lib/milvus
```

> 📌 **解释：** 这里的配置确保你的向量数据库数据存储在宿主机的硬盘上。
>
> ✅ **安全性：** 即使你执行 `docker compose down` 删除了容器，你的政策文档向量数据也不会丢失，下次启动时会自动挂载。

12.3.5 安全与性能配置 (`security_opt`)

`seccomp:unconfined`：

- **原理：** Milvus 在执行高性能向量计算时，需要直接调用一些底层系统指令
- **设置意义：** 取消默认的安全计算模型限制，可以提升向量检索的性能，并减少由于权限拦截导致的诡异报错

12.3.6 4GB 内存服务器的"生存法则"

通过这份配置，你的 4GB 内存被精确地切分成了几块"自留地"：

组件	占用	优先级
操作系统	留存 ~0.4GB	🔴 必须
Milvus 全家桶	占用 ~1.2GB（1024M 主程序 + 辅助组件）	🔴 必须
Spring Boot	建议限制在 0.6GB (-Xmx512m)	🔴 必须
Ollama (模型权重)	剩余约 1.8GB - 2.0GB	🔴 必须

12.4 应用新配置步骤

```
# 停止并移除旧容器
docker compose down

# 应用新配置并以后台模式启动
docker compose up -d

# 观察实时内存占用（关键）
docker stats
```

✅ **检查点：** 确保 `milvus-standalone` 的 `MEM USAGE` 稳定在 900MB-1000MB 之间。如果你看到 `milvus-standalone` 的 `MEM USAGE` 稳定在 900MB - 1000MB 之间，且 `milvus-minio` 在 200MB 左右，说明配置生效，你的系统处于 **最稳健** 的运行状态。

12.5 解决端口冲突

如果遇到 `9000 failed: port is already allocated`，通常是 Portainer 占用了端口。建议卸载以节省约 100MB 内存：

```
docker stop portainer && docker rm portainer
docker volume rm portainer_data
```

第 13 章 Ollama 模型管理与调优

13.1 Ollama 安装与配置

13.1.1 一键安装脚本

```
curl -fsSL https://ollama.com/install.sh | sh
```

13.1.2 常见报错处理

报错	处理方案	是否可忽略
`nvidia-smi` 未找到	若非 GPU 实例，此报错可忽略	✅ 是
`aplay command not found`	阿里云精简镜像缺少组件。执行 `apt update && apt install alsa-utils -y` 修复	❌ 否

13.1.3 远程访问配置

若需跨机器调用 API，需执行：

```
sudo systemctl edit ollama.service
# 添加以下内容：
[Service]
Environment="OLLAMA_HOST=0.0.0.0"
# 重启服务
sudo systemctl daemon-reload && sudo systemctl restart ollama
```

✅ **终极验证：** 浏览器中输入 `http://阿里云公网 ip:11434/api/tags` 应显示模型列表 JSON。

13.2 拉取模型（按路线选择）

🌟 Path A (标准配置)

```
ollama pull qwen:7b   # 对话大模型
ollama pull bge-m3    # 向量模型 (维度 1024)
```

💻 Path B (低配配置)

```
ollama pull qwen2.5:1.5b
ollama pull bge-m3    # 强烈推荐统一使用 bge-m3
```

> ⚠ **注意：** `bge-small-zh` 官方库不存在，推荐直接使用 `bge-m3`。即使是低配机器，bge-m3 多占的几百 MB 内存也是值得的，能省去大量手动配置麻烦且精度更高。

13.3 BGE 向量模型避坑

执行 `ollama pull bge-small-zh` 若报 `file does not exist`，是因为官方库名称不匹配。

推荐替代方案： 使用 BGE-M3（支持 80+ 语言，含中文最佳）。

```
ollama pull bge-m3
```

手动导入特定模型：

1. 下载 `.gguf` 文件
2. 创建 `Modelfile` 写入 `FROM ./xxx.gguf`
3. 执行 `ollama create <自定义名> -f Modelfile`

手动导入 BGE-Small-ZH (极限低配 Path B)：

```
# 下载 GGUF
wget https://huggingface.co/C-K-L/bge-small-zh-v1.5-GGUF/resolve/main/bge-small-zh-v1.5-q4_k_m.gguf -O bge-small.gguf

# 创建 Modelfile
cat > Modelfile << EOF
FROM ./bge-small.gguf
TEMPLATE ""
PARAMETER num_ctx 512
EOF

# 构建
ollama create bge-small-zh -f Modelfile
```

13.4 强制限制推理线程 (关键优化)

> ⚠ **默认 Ollama 会尝试占满所有核心。限制其使用单线程，留出核心处理业务**

```
sudo systemctl edit ollama.service
# 添加以下内容
[Service]
Environment="OLLAMA_NUM_THREAD=1"       # 强制模型只使用 1 个线程
Environment="OLLAMA_KEEP_ALIVE=5m"      # 5 分钟不使用自动释放内存
Environment="OLLAMA_NUM_PARALLEL=1"     # 串行处理防止瞬时崩溃
Environment="OLLAMA_HOST=0.0.0.0"       # 允许内网跨容器访问
sudo systemctl daemon-reload && sudo systemctl restart ollama
```

13.5 模型选型与预热

推荐模型：

- `qwen2.5:0.5b`（推理极快，400MB 内存占用）
- `qwen2.5:1.5b`（1.1GB 占用，理解力尚可）

预热**技巧：** 防止首次搜索超时。

```
# 预热 Embedding 模型 (BGE-M3)
curl http://localhost:11434/api/embeddings -d '{
    "model": "bge-m3",
    "prompt": "warmup"
}'

# 预热 Chat 模型 (Qwen2.5-1.5B)
curl http://localhost:11434/api/chat -d '{
    "model": "qwen2.5:1.5b",
    "messages": [{"role": "user", "content": "hi"}]
}'
```

13.6 阿里云/腾讯云安全组提醒

> ⚠ **请务必在云控制台的"安全组 - 入方向规则"中，放行以下端口**

端口	服务	协议	授权对象
11434	Ollama	TCP	0.0.0.0/0
19530	Milvus	TCP	0.0.0.0/0
8080	Java API	TCP	0.0.0.0/0
9000/9001	Portainer/MinIO	TCP	你的 IP/32

第 14 章 可视化与监控工具

14.1 Portainer 安装与管理

Portainer 提供网页界面管理容器。注意：其有 5-10 分钟的安全保护机制。

14.1.1 安装命令

```
docker run -d -p 9000:9000 --name portainer --restart=always \
  -v /var/run/docker.sock:/var/run/docker.sock \
  portainer/portainer-ce
```

14.1.2 无法访问/超时排查

现象： 提示安全超时，无法设置密码。

对策： 重启容器以重置计时器：

```
docker restart portainer
```

✅ **后续：** 立即访问 `http://服务器 IP:9000` 设置 12 位以上密码。

14.2 Attu 可视化工具

> ⚠ **避坑：** 严禁在服务器上额外部署 Attu 可视化镜像。请下载 Attu 桌面客户端，通过阿里云公网 IP 远程连接 Milvus (19530)。

14.3 内存监控

```
# 实时监控物理内存
watch -n 1 free -h

# 查看容器资源占用
docker stats
```

健康指标

指标	健康范围	说明	告警阈值
Used	3.2G ~ 3.6G	最理想状态，说明资源被充分利用且没有溢出	> 3.8G
Available	≥150MB-200MB	系统就不会触发 OOM Killer 杀掉进程	< 100MB
Swap	1G-2G	占用是正常现象	> 4G

第四部分：后端开发实战 (Spring AI)

第 15 章 项目初始化与依赖冲突解决

15.1 终极兼容版 pom.xml

> 🔥 **避坑解析：** 之前出现的 `TypeTag :: UNKNOWN` 报错，根本原因是 Maven 3.9+ 对 XML 标签闭合要求极其严格，且旧版 Lombok 无法解析 JDK21 字节码。请完全替换你的 POM 文件，并在 IDEA 中 Reload。

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
   <modelVersion>4.0.0</modelVersion>

   <parent>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-parent</artifactId>
      <version>3.4.3</version>
      <relativePath/>
   </parent>

   <groupId>com.ailearn.governmentaffairsrag</groupId>
   <artifactId>spring-ai-policy-rag-system</artifactId>
   <version>1.0-SNAPSHOT</version>

   <properties>
      <java.version>21</java.version>
      <!-- ⚠ 核心修复：锁定兼容 JDK 21 的 Lombok 版本 -->
      <lombok.version>1.18.36</lombok.version>
      <spring-ai.version>1.0.0-M6</spring-ai.version>
   </properties>

   <dependencyManagement>
      <dependencies>
         <dependency>
            <groupId>org.springframework.ai</groupId>
            <artifactId>spring-ai-bom</artifactId>
            <version>${spring-ai.version}</version>
            <type>pom</type>
            <scope>import</scope>
         </dependency>
      </dependencies>
   </dependencyManagement>

   <dependencies>
      <!-- Web & 流式响应 -->
      <dependency>
         <groupId>org.springframework.boot</groupId>
         <artifactId>spring-boot-starter-web</artifactId>
      </dependency>
      <dependency>
         <groupId>org.springframework.boot</groupId>
         <artifactId>spring-boot-starter-webflux</artifactId>
```

```
        </dependency>

        <!-- Spring AI 核心组件 -->
        <dependency>
            <groupId>org.springframework.ai</groupId>
            <artifactId>spring-ai-ollama-spring-boot-starter</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.ai</groupId>
            <artifactId>spring-ai-milvus-store-spring-boot-starter</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.ai</groupId>
            <artifactId>spring-ai-tika-document-reader</artifactId>
        </dependency>

        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <version>${lombok.version}</version>
            <scope>provided</scope>
        </dependency>
    </dependencies>

    <!-- ⚠ 必加：Spring AI M 版依赖的里程碑仓库 -->
    <repositories>
        <repository>
            <id>spring-milestones</id>
            <name>Spring Milestones</name>
            <url>https://repo.spring.io/milestone</url>
            <snapshots><enabled>false</enabled></snapshots>
        </repository>
    </repositories>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.13.0</version>
                <configuration>
                    <source>${java.version}</source>
                    <target>${java.version}</target>
                    <annotationProcessorPaths>
                        <path>
                            <groupId>org.projectlombok</groupId>
                            <artifactId>lombok</artifactId>
                            <version>${lombok.version}</version>
                        </path>
                    </annotationProcessorPaths>
                </configuration>
            </plugin>
        </plugins>
```

```
    </build>
</project>
```

15.2 替换 POM 后的关键操作

Windows 用户请执行：

```
rmdir /s /q "D:\Program Files\maven\repository\org\projectlombok\lombok"
```

清除旧缓存，然后执行 `mvn clean install -U` 强制刷新。

15.3 项目结构推荐

```
my-project/
├── docker-compose.yml     # 核心编排文件（包含 Milvus、Etcd、MinIO、JavaApp）
└── java-app/
    ├── Dockerfile         # Java 镜像定义
    └── app.jar            # 编译好的 Java 程序
```

15.4 Java 服务 Dockerfile 示例

```
FROM openjdk:21-jdk-slim
WORKDIR /app
COPY target/app.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-Xms512m", "-Xmx600m", "-jar", "app.jar"]
```

第 16 章 核心配置与自动装配陷阱

16.1 application.yml 配置文件

> ⚠ **极其重要：** `embedding-dimension` 必须与你拉取的模型对应！`bge-m3` = 1024，`bge-small` = 512。

```
spring:
  application:
    name: policy-rag-system
  ai:
    ollama:
      base-url: http://<你的阿里云公网 IP>:11434  # 本地开发替换为云端 IP
      chat:
        model: qwen:7b   # Path B 用户改为 qwen2.5:1.5b
        options:
```

```yaml
                temperature: 0.3  # 政策问答需严谨，降低发散性
            embedding:
                model: bge-m3    # 强烈推荐统一使用 bge-m3
        vectorstore:
            milvus:
                client:
                    host: <你的阿里云公网 IP>
                    port: 19530
                collection-name: policy_docs
                embedding-dimension: 1024  # ⚠ 匹配 bge-m3 的维度
                index-type: HNSW           # 使用 HNSW 保证检索速度
    mvc:
        async:
            request-timeout: 300000  # 5 分钟异步超时，防止 AI 响应慢断开

logging:
    level:
        com.ailearn.rag: DEBUG
        org.springframework.ai: DEBUG  # 开启日志以查看真实 Prompt
    file:
        name: logs/rag.log
    logback:
        rollingpolicy:
            file-name-pattern: logs/rag-%d{yyyy-MM-dd}.%i.log
            max-file-size: 100MB
```

16.2 Spring Boot 启动调优 (JVM 限制)

> ⚠ **禁止 Java 无节制占用物理内存**

```bash
# 限制最大堆内存，预留空间给 Ollama 推理
java -Xms512m -Xmx600m -XX:MaxMetaspaceSize=256m -jar policy-rag-system.jar
```

Path B 低配机器建议：

```bash
java -Xms1g -Xmx2g -XX:+UseG1GC -jar government-qa-system.jar
```

16.3 Spring AI 核心配置调优 (1.0.0-M6 版)

> 🔧 **解决报错：** `io.milvus.exception.ServerException: collection not found[database=default][collection=vector_store]`

在 M6 版本中，自动配置存在不稳定性，常导致 `collection-name` 失效（报错寻找默认的 `vector_store`）。

16.3.1 强制手动配置方案 (VectorStoreConfig)

由于 YAML 绑定可能失效，建议通过 Java 配置类强行接管主权。

技术点： 注入官方源码类 `MilvusVectorStoreProperties`，并开启属性绑定。

必杀技： 显式设置 `.initializeSchema(true)` 解决"集合不存在"导致的搜索崩溃。

```
@Configuration
@EnableConfigurationProperties(MilvusVectorStoreProperties.class)
public class VectorStoreConfig {
   @Bean
   @Primary
   public MilvusVectorStore vectorStore(MilvusServiceClient client,
           EmbeddingModel model,
           MilvusVectorStoreProperties properties) {
      return MilvusVectorStore.builder(client, model)
              .collectionName(properties.getCollectionName())
              .embeddingDimension(properties.getEmbeddingDimension())
              .initializeSchema(true)  // 启动时自动检查/创建集合
              .build();
   }
}
```

16.3.2 YAML 配置对齐

确保路径严格匹配 `spring.ai.vectorstore.milvus`（注意 M6 以后通常不带横杠）。

```
spring:
   ai:
      ollama:
         base-url: http://服务器 IP:11434
      vectorstore:
         milvus:
            client:
               host: 服务器 IP
               port: 19530
            collection-name: policy_docs
            embedding-dimension: 1024  # 必须与 BGE-M3 一致
```

第 17 章 文档解析与分批向量化

17.1 文档解析与入库服务 (IngestionService.java)

```
@Service
@RequiredArgsConstructor
public class IngestionService {
   private final VectorStore vectorStore;
```

```
    public void processDocument(MultipartFile file) throws IOException {
        // 1. Tika 万能文档解析
        TikaDocumentReader loader = new TikaDocumentReader(
                new InputStreamResource(file.getInputStream())
        );

        // 2. Token 智能切片 (保留上下文重叠度，防止语义截断)
        TextSplitter splitter = new TokenTextSplitter(500, 100, 10, 10000, true);
        List<Document> splitDocuments = splitter.apply(loader.get());

        // 3. 注入文件名元数据，用于追溯来源
        splitDocuments.forEach(doc ->
                doc.getMetadata().put("filename", file.getOriginalFilename())
        );

        // 4. 向量化并持久化至 Milvus
        vectorStore.add(splitDocuments);
    }
}
```

17.2 高效 Ingestion（分批入库逻辑）

> ⚠ **避免一次性处理大文件导致 OOM**

```
public void processDocument(MultipartFile file) throws IOException {
    TikaDocumentReader loader = new TikaDocumentReader(
            new InputStreamResource(file.getInputStream())
    );
    TokenTextSplitter splitter = new TokenTextSplitter(400, 100, 5, 10000, true);
    List<Document> docs = splitter.apply(loader.get());

    // 每批 32 条最稳健
    int batchSize = 32;
    for (int i = 0; i < docs.size(); i += batchSize) {
        vectorStore.add(docs.subList(i, Math.min(i + batchSize, docs.size())));
    }
}
```

17.3 维度冲突处理

> ⚠ **关键：** 向量维度 (Dimension) 必须严格匹配！

如果你更换了 Embedding 模型，必须删除并重建 Milvus 的 Collection。512 维与 1024 维不匹配会导致检索时抛出异常。

第 18 章 RAG 检索与 SSE 流式响应

18.1 RAG 流式问答服务 (RagService.java)

```java
package com.wx.rag.service;

import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.document.Document;
import org.springframework.ai.vectorstore.SearchRequest;
import org.springframework.ai.vectorstore.VectorStore;
import org.springframework.stereotype.Service;
import reactor.core.publisher.Flux;

import java.util.List;
import java.util.stream.Collectors;

@Service
public class RagService {

   private final ChatClient chatClient;
   private final VectorStore vectorStore;

   // 推荐在构造时直接构建好 ChatClient 实例
   public RagService(ChatClient.Builder chatClientBuilder, VectorStore
vectorStore) {
      this.chatClient = chatClientBuilder.build();
      this.vectorStore = vectorStore;
   }

   public Flux<String> streamAnswer(String query) {

      // 1. 向量数据库相似度检索 (Top 3，阈值 0.6)
      // 使用新版的 Builder 模式
      List<Document> docs = vectorStore.similaritySearch(

SearchRequest.builder().query(query).topK(3).similarityThreshold(0.6).build());

      // 2. 组装上下文与引用来源
      String context = docs.stream().map(Document::getText)    // ✅ 改为
getText() 即可
              .collect(Collectors.joining("\n\n"));

      String refs = docs.stream().map(d ->
(String)d.getMetadata().getOrDefault("filename", "未知来源")).distinct()
              .collect(Collectors.joining("，  "));

      // 3. 构建 Prompt
      String prompt = "基于以下政策上下文回答问题：\n" + context + "\n\n问题：" +
query;

      // 4. 流式 (SSE) 返回内容，并在末尾拼接来源参考
      // 新版 ChatClient 推荐使用 .prompt().user(prompt) 语法
      return
chatClient.prompt().user(prompt).stream().content().concatWith(Flux.just("\n\n---
\n📚 来源：" + refs));
   }
}
```

18.2 控制层 (ChatController.java)

```
@RestController
@RequestMapping("/api")
@CrossOrigin(origins = "*")  // 允许前端跨域
@RequiredArgsConstructor
public class ChatController {

    private final RagService ragService;
    private final IngestionService ingestionService;

    // 流式问答接口 (SSE)
    @GetMapping(value = "/chat", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    public Flux<String> chat(@RequestParam String query) {
        return ragService.streamAnswer(query);
    }

    // 文档上传接口
    @PostMapping("/upload")
    public String upload(@RequestParam("file") MultipartFile file) throws
IOException {
        ingestionService.processDocument(file);
        return "上传并解析成功：" + file.getOriginalFilename();
    }
}
```

18.3 性能优化：如何达到 2s 响应？

优化项	具体措施	预期提升	实现难度
模型量化	在 Ollama 中务必使用 q4 或 q5 量化版本，减少显存占用并提升生成速度	30-50%	☆
Milvus 索引优化	为向量字段创建 HNSW 索引，M 值设为 16，efConstruction 设为 128，可实现毫秒级检索	50-70%	☆☆
JVM 调优	针对 Java 服务，设置 -Xmx16g（根据内存实际情况），避免在高并发解析 PDF 时触发频繁 GC	20-30%	☆☆
前端 SSE 渲染	Vue3 使用 EventSource 接收数据，避免等待模型生成完整段落，提升用户 "首字" 体感速度	体感 50%+	☆☆

第 19 章 完整代码汇总 (Controller/Service/Config)

19.1 完整项目结构

```
spring-ai-policy-rag-system/
├── pom.xml
├── src/
│   ├── main/
│   │   ├── java/com/ailearn/rag/
│   │   │   ├── PolicyRagApplication.java
│   │   │   ├── config/
│   │   │   │   ├── VectorStoreConfig.java
│   │   │   │   └── CorsConfig.java
│   │   │   ├── controller/
│   │   │   │   └── ChatController.java
│   │   │   ├── service/
│   │   │   │   ├── RagService.java
│   │   │   │   └── IngestionService.java
│   │   │   └── dto/
│   │   │       └── ChatRequest.java
│   │   └── resources/
│   │       ├── application.yml
│   │       └── logback-spring.xml
│   └── test/
├── docker-compose.yml
└── Dockerfile
```

19.2 主启动类 (PolicyRagApplication.java)

```java
package com.ailearn.rag;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class PolicyRagApplication {
   public static void main(String[] args) {
      SpringApplication.run(PolicyRagApplication.class, args);
   }
}
```

19.3 跨域配置 (CorsConfig.java)

```java
package com.ailearn.rag.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class CorsConfig implements WebMvcConfigurer {
   @Override
```

```
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**")
                .allowedOrigins("*")
                .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
                .allowedHeaders("*")
                .maxAge(3600);
    }
}
```

19.4 完整 VectorStoreConfig.java

```
package com.ailearn.rag.config;

import io.milvus.client.MilvusServiceClient;
import org.springframework.ai.embedding.EmbeddingModel;
import org.springframework.ai.vectorstore.MilvusVectorStore;
import org.springframework.ai.vectorstore.MilvusVectorStoreProperties;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Primary;
import org.springframework.boot.context.properties.EnableConfigurationProperties;

@Configuration
@EnableConfigurationProperties(MilvusVectorStoreProperties.class)
public class VectorStoreConfig {

    @Bean
    @Primary
    public MilvusVectorStore vectorStore(MilvusServiceClient client,
            EmbeddingModel model,
            MilvusVectorStoreProperties properties) {
        return MilvusVectorStore.builder(client, model)
                .collectionName(properties.getCollectionName())
                .embeddingDimension(properties.getEmbeddingDimension())
                .initializeSchema(true)
                .build();
    }
}
```

19.5 完整 logback-spring.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
    <include resource="org/springframework/boot/logging/logback/base.xml"/>

    <appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
        <file>logs/rag.log</file>
        <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
            <fileNamePattern>logs/rag-%d{yyyy-MM-dd}.%i.log</fileNamePattern>
```

```
            <timeBasedFileNamingAndTriggeringPolicy
class="ch.qos.logback.core.rolling.SizeAndTimeBasedFNATP">
                <maxFileSize>100MB</maxFileSize>
            </timeBasedFileNamingAndTriggeringPolicy>
            <maxHistory>30</maxHistory>
        </rollingPolicy>
        <encoder>
            <pattern>%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36} -
%msg%n</pattern>
        </encoder>
    </appender>

    <logger name="com.ailearn.rag" level="DEBUG"/>
    <logger name="org.springframework.ai" level="DEBUG"/>

    <root level="INFO">
        <appender-ref ref="FILE"/>
        <appender-ref ref="CONSOLE"/>
    </root>
</configuration>
```

第 20 章 RAG 检索优化 (混合检索/Rerank)

20.1 混合检索（Hybrid Search）

单一向量检索的局限：

- ❌ 无法处理精确关键词匹配（如产品型号、政策编号）
- ❌ 对专业术语敏感度低

混合检索方案：

```
@Service
public class HybridSearchService {

    private final VectorStore vectorStore;
    private final MilvusServiceClient milvusClient;

    // 混合检索：向量相似度 + 关键词匹配
    public List<Document> hybridSearch(String query, String keywordFilter, int
topK) {
        // 1. 向量检索
        List<Document> vectorResults = vectorStore.similaritySearch(
                SearchRequest.builder()
                        .query(query)
                        .topK(topK * 2)  // 先取更多候选
                        .similarityThreshold(0.5)
                        .build()
        );
```

```
        // 2. 关键词过滤（标量字段）
        return vectorResults.stream()
                .filter(doc -> {
                    String text = doc.getText();
                    return text.contains(keywordFilter);  // 简单关键词匹配
                })
                .limit(topK)
                .collect(Collectors.toList());
    }
}
```

20.2 Rerank 重排序优化

为什么需要 Rerank？

- 初步检索返回的 Top-K 可能包含不相关文档
- Rerank 模型可对候选文档进行精细排序

集成 Rerank 模型：

```
@Service
public class RerankService {

    private final RestTemplate restTemplate;

    // 使用 BGE-Reranker 进行重排序
    public List<Document> rerank(String query, List<Document> candidates) {
        // 1. 准备请求数据
        List<String> texts = candidates.stream()
                .map(Document::getText)
                .collect(Collectors.toList());

        // 2. 调用 Rerank API（本地部署或云端）
        RerankRequest request = new RerankRequest(query, texts);
        RerankResponse response = restTemplate.postForObject(
                "http://localhost:8888/rerank",
                request,
                RerankResponse.class
        );

        // 3. 按分数重新排序
        return response.getResults().stream()

.sorted(Comparator.comparingDouble(RerankResult::getScore).reversed())
                .map(result -> candidates.get(result.getIndex()))
                .limit(3)  // 只保留 Top 3
                .collect(Collectors.toList());
    }
}
```

Rerank 模型推荐：

模型	参数量	精度提升	延迟	内存占用	推荐场景
bge-reranker-large	335M	+15%	100ms	~800MB	生产环境
bge-reranker-base	118M	+12%	50ms	~400MB	平衡方案
bge-reranker-small	33M	+8%	20ms	~150MB	低配环境

20.3 查询重写（Query Rewriting）

问题： 用户提问往往不够精确，影响检索效果。

解决方案： 使用 LLM 重写查询，扩展语义。

```
@Service
public class QueryRewriteService {

    private final ChatClient chatClient;

    // 查询重写：扩展语义、同义词替换
    public String rewriteQuery(String originalQuery) {
        String prompt = """
                请对以下用户问题进行改写，使其更适合向量检索：
                1. 扩展同义词
                2. 补充隐含信息
                3. 保持核心语义不变

                原问题：%s

                改写后的问题：
                """.formatted(originalQuery);

        return chatClient.prompt()
                .user(prompt)
                .call()
                .content();
    }
}
```

示例：

- **原问题：** "人才补贴怎么申请？"
- **改写后：** "人才购房补贴申请条件 人才补贴申请流程 政府人才补助政策"

20.4 检索性能优化检查清单

优化项	检查点	目标值	优先级

优化项	检查点	目标值	优先级
索引类型	是否使用 HNSW	M=16, efConstruction=128	🔴 高
向量维度	是否与模型匹配	bge-m3=1024, bge-small=512	🔴 高
Top-K 设置	是否合理	5-10（过大影响性能）	🟡 中
相似度阈值	是否过滤低质量	0.5-0.7	🟡 中
缓存策略	是否启用查询缓存	热点查询缓存命中率 >80%	🟢 低
并发控制	是否限制并发检索	单实例 <10 QPS	🟡 中

第 21 章 RAG 系统评估体系

21.1 核心评估指标

指标	定义	计算公式	目标值	测量方法
检索召回率 (Recall)	检索到的相关文档占比	相关检索结果/总相关文档	>85%	人工标注测试集
检索准确率 (Precision)	检索结果中相关的占比	相关检索结果/总检索结果	>80%	人工标注测试集
答案准确率 (Answer Accuracy)	生成答案的正确性	正确回答数/总问题数	>90%	LLM 裁判或人工
首字延迟 (TTFT)	从提问到看到第一个字的时间	-	<2s	前端埋点
完整响应时间	从提问到完整答案的时间	-	<10s	前端埋点
幻觉率 (Hallucination)	生成虚假信息的比例	幻觉回答数/总回答数	<5%	人工审核

21.2 自动化评估脚本

```
@Service
public class RagEvaluationService {

    private final VectorStore vectorStore;
    private final ChatClient chatClient;

    // 评估检索质量
    public EvaluationResult evaluateRetrieval(List<EvaluationCase> testCases) {
        int totalRelevant = 0;
        int totalRetrieved = 0;

        for (EvaluationCase testCase : testCases) {
            List<Document> results = vectorStore.similaritySearch(
                    SearchRequest.query(testCase.getQuery()).topK(5)
```

```
            );

            // 检查是否检索到预期文档
            boolean found = results.stream()
                    .anyMatch(doc -> doc.getMetadata()
                            .containsKey(testCase.getExpectedDocId()));

            if (found) totalRelevant++;
            totalRetrieved++;
        }

        return EvaluationResult.builder()
                .recall((double) totalRelevant / testCases.size())
                .precision((double) totalRelevant / totalRetrieved)
                .build();
    }

    // 评估答案质量（使用 LLM 作为裁判）
    public AnswerQuality evaluateAnswer(String question, String answer, String
groundTruth) {
        String prompt = """
                请评估以下 AI 回答的质量：

                问题：%s
                AI 回答：%s
                标准答案：%s

                请从以下维度评分（0-10 分）：
                1. 准确性
                2. 完整性
                3. 相关性

                以 JSON 格式返回评分。
                """.formatted(question, answer, groundTruth);

        String evaluation = chatClient.prompt()
                .user(prompt)
                .call()
                .content();

        return parseEvaluationJson(evaluation);
    }
}
```

21.3 评估数据集构建

测试用例模板：

```
{
    "testCases": [
        {
```

```
            "id": "case_001",
            "query": "人才购房补贴申请条件是什么？",
            "expectedDocIds": ["policy_2024_001", "policy_2024_003"],
            "groundTruth": "申请人需满足：1. 本科及以上学历 2. 本地缴纳社保满 12 个月 3.
首次购房...",
            "category": "人才政策"
        },
        {
            "id": "case_002",
            "query": "企业研发费用加计扣除比例是多少？",
            "expectedDocIds": ["tax_2024_005"],
            "groundTruth": "制造业企业研发费用加计扣除比例为 100%...",
            "category": "税收政策"
        }
    ]
}
```

> 💡 **建议：** 至少准备 50-100 个测试用例，覆盖所有业务场景。

第五部分：前端交互与体验优化

第 22 章 Vue3 项目搭建

22.1 项目创建

```
npm create vue@latest  # 推荐 Vite
cd <project-name> && npm install
npm install @microsoft/fetch-event-source  # 专业流解析库
npm install markdown-it  # Markdown 渲染
```

22.2 项目结构

```
vue-app/
├── src/
│   ├── components/
│   │   └── ChatBox.vue
│   ├── App.vue
│   └── main.js
├── package.json
└── vite.config.js
```

第 23 章 SSE 流式通信原生实现

23.1 流式响应解析 (原生 Fetch)

> ⚠ **避免第三方库的自动重连机制在 500 错误时压死服务器**

```
const sendMessage = async (query) => {
   if (!userInput.value.trim()) return

   const messages = ref([{ role: 'user', content: query }])
   const assistantMsg = { role: 'assistant', content: '' }
   messages.value.push(assistantMsg)

   try {
      const response = await fetch(
              `http://localhost:8080/api/chat?query=${encodeURIComponent(query)}`,
              { signal: abortController.signal }
      )
      const reader = response.body.getReader()
      const decoder = new TextDecoder()

      while (true) {
         const { done, value } = await reader.read()
         if (done) break
         const chunk = decoder.decode(value, { stream: true })
         // 处理 SSE 格式中的 "data:" 前缀
         assistantMsg.content += chunk.replace(/^data:/gm, '')
      }
   } catch (error) {
      assistantMsg.content += `[系统错误：响应中断${error}]`
   }
}
```

23.2 专业流式解析逻辑（使用 fetch-event-source）

✅ **避免手动解析 `reader.read()` 产生的 SSE 协议头（如 `data:` 字符）**

```
import { fetchEventSource } from '@microsoft/fetch-event-source';

const sendMessage = async (query) => {
   const assistantMsg = { role: 'assistant', content: '' };
   messages.value.push(assistantMsg);

   await fetchEventSource('/api/chat/stream', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ query }),
      onmessage(ev) {
         if (ev.data) assistantMsg.content += ev.data;
      },
      onclose() { /* 处理关闭 */ },
      onerror(err) { throw err; }
   });
};
```

第 24 章 UI/UX 优化与 Markdown 渲染

24.1 完整 Vue3 组件示例

```
<template>
   <div class="chat-container">
      <div class="messages" ref="msgBox">
         <div v-for="(msg, index) in messages" :key="index" :class="['message',
msg.role]">
            <div class="content" v-html="renderMarkdown(msg.content)"></div>
         </div>
      </div>

      <div class="input-area">
         <input
                v-model="userInput"
                @keyup.enter="sendMessage"
                placeholder="请输入政策问题..."
                :disabled="loading"
         />
         <button @click="sendMessage" :disabled="loading">发送</button>
      </div>
   </div>
</template>

<script setup lang="ts">
   import { ref } from 'vue'
   import MarkdownIt from 'markdown-it'

   const md = new MarkdownIt()
   const userInput = ref('')
   const messages = ref<{ role: string; content: string }[]>([])
   const loading = ref(false)

   const renderMarkdown = (text: string) => {
      return md.render(text)
   }

   const sendMessage = async () => {
      if (!userInput.value.trim()) return

      const query = userInput.value
      messages.value.push({ role: 'user', content: query })
      userInput.value = ''
      loading.value = true

      // 预占位 Assistant 的回复
      const assistantMsg = { role: 'assistant', content: '' }
      messages.value.push(assistantMsg)
```

```
        try {
           const response = await fetch(
                   `http://localhost:8080/api/chat/stream?
query=${encodeURIComponent(query)}`,
           )
           const reader = response.body!.getReader()
           const decoder = new TextDecoder()

           while (true) {
              const { done, value } = await reader.read()
              if (done) break
              const chunk = decoder.decode(value, { stream: true })
              // 实时追加内容
              assistantMsg.content += chunk
           }
        } catch (error) {
           assistantMsg.content += `[系统错误：响应中断${error}]`
        } finally {
           loading.value = false
        }
     }
  </script>

  <style scoped>
     .chat-container {
        max-width: 800px;
        margin: 0 auto;
        padding: 20px;
     }
     .message {
        margin-bottom: 15px;
        padding: 10px;
        border-radius: 8px;
     }
     .user {
        background-color: #e3f2fd;
        text-align: right;
     }
     .assistant {
        background-color: #f5f5f5;
     }
  </style>
```

24.2 UI/UX 优化建议

优化项	实现方式	用户体验提升	实现难度
Markdown 渲染	使用 markdown-it 处理标题、表格	答案更易读	☆
思考时长计时	通过首字下行（TTFT）计算 RAG 检索效率	用户知道系统在工作	☆☆
引用来源展示	在回答末尾拼接来源文件信息	增加答案可信度	☆

优化项	实现方式	用户体验提升	实现难度
流式输出	使用 SSE 实现打字机效果，提升用户体感速度	减少等待焦虑	☆☆

第六部分：低配环境极限生存指南 (重点)

第 25 章 2 核 4G 服务器生存法则

25.1 2G 环境下的表现与对策

组件	2G 环境下的表现	2G 生存级对策	可行性
Ollama (LLM)	频繁触发 Swap，响应需数分钟	降级至 `qwen2.5:0.5b` (约 397MB)	✅ 可行
Ollama (Embed)	直接导致 OOM (内存溢出)	换用 `bge-small-zh` 或 `all-minilm`	✅ 可行
Milvus	容器因内存不足频繁重启	严苛限制 Docker 内存配额	✅ 可行
Spring Boot	启动即卡死，无法分配内存	极度限制堆内存 `-Xmx256m`	⚠ 风险高

25.2 个人实验环境的"极限调优"（无 GPU 必看）

> ⚠ **如果你使用的是普通的双核或四核 ECS，必须通过以下手段保证 2s 左右的响应**

25.2.1 模型降级：以小博大 (最有效)

> 🚫 **严禁在 CPU 环境下强跑 Qwen-7B。** 为了流畅度，请使用以下模型：
>
> - **Qwen2.5-1.5B：** 个人实验的"神级"选择，理解力尚可，CPU 运行极快（15+ tokens/s）
> - **DeepSeek-R1-Distill-Qwen-1.5B：** 适合需要逻辑推理的政策问答

```
ollama run qwen2.5:1.5b
```

25.2.2 内存救命药：开启 Swap

云服务器内存通常较小，务必手动分配虚拟内存防止服务被系统杀掉：

```
sudo fallocate -l 8G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
```

25.2.3 Milvus 瘦身

在 `docker-compose.yml` 中，限制 Milvus 容器的内存使用（例如限制在 2G 内存内），为 Spring Boot 和 Ollama 留出空间。

25.2.4 JVM 参数调优

Java 进程建议使用 G1 垃圾回收器，限制堆内存大小：

```
java -Xms1g -Xmx2g -XX:+UseG1GC -jar government-qa-system.jar
```

25.3 实验配置总结表

维度	推荐配置 (个人实验版)	预期表现	优先级
ECS 规格	2 核 4G 或 4 核 8G	勉强维持多组件运行	⚫ 必须
模型选择	Qwen2.5-1.5B	响应延迟 < 2s	⚫ 必须
关键技术	开启 8G Swap	系统不卡死、不重启	⚫ 必须
向量库	Milvus Standalone	检索延迟 < 50ms	⚫ 必须
交互方式	流式输出 (SSE)	用户体感极佳	⚪ 推荐

> 💡 **结语：** 在没有 GPU 的个人实验场景下，不要追求模型的"参数大"，而要追求系统的"链路通"。使用 Qwen2.5-1.5B 配合 Swap 虚拟内存，你完全可以在一台普通的阿里云服务器上构建出一个极其流畅的政务 AI 助手。

第 26 章 内存管理与 Swap 调优

26.1 4GB 内存的黄金配置

✅ **升级到 4G 后，你已跨过生存线。建议采用 BGE-M3 (高精检索) + Qwen-1.5B (智能对话) 的黄金组合**

26.1.1 内存资产负债表 (合理分配 4GB)

为防止某个进程"暴饮暴食"，必须划定红线：

组件	分配	监控命令	告警阈值
Ollama (模型权重)	约 2.2 GB (加载 BGE-M3 + Qwen-1.5B)	ollama ps	> 2.5G
Milvus (向量数据库)	约 0.8 GB (通过 Docker 限制)	docker stats	> 1.2G
Spring Boot (JVM)	约 0.6 GB (通过启动参数限制)	jstat -gc	> 1G
OS 预留 (系统底层)	约 0.4 GB	free -h	< 200MB
[防线] Swap	保持开启 8GB Swap，作为应对突发峰值的缓冲	swapon -s	> 4G

26.2 Ollama 服务管理 (防内存常驻)

编辑 Ollama 配置，确保模型在不使用时及时释放内存：

```
sudo SYSTEMD_EDITOR=vim systemctl edit ollama.service
```

在 `[Service]` 块中添加：

```
[Service]
Environment="OLLAMA_KEEP_ALIVE=5m"    # 5 分钟不使用自动释放内存
Environment="OLLAMA_NUM_PARALLEL=1"    # 串行处理，防止并发撑爆内存
```

重启服务：

```
sudo systemctl daemon-reload && sudo systemctl restart ollama
```

第 27 章 组件资源配额化实战

27.1 最终内存动态监控

打开一个新窗口，输入以下命令实时监控物理内存：

```
watch -n 1 free -h
```

健康指标

指标	健康范围	说明	告警动作
Used	3.2G ~ 3.6G	这是最理想的状态，说明资源被充分利用且没有溢出	> 3.8G 时告警
Available	≥150MB	系统就不会触发 OOM Killer 杀掉进程	< 100MB 时清理缓存

27.2 最终健康度监控

执行 `free -h`：

指标	安全范围	说明	处理建议
Available	> 200MB	系统就是安全的	保持监控
Swap	1G-2G	占用是正常现象	> 4G 时需优化

27.3 每日凌晨例行维护脚本

创建 `daily_maint.sh` 并加入 crontab：

```
#!/bin/bash
# 1. 重启 Ollama 释放内存碎片
systemctl restart ollama
# 2. 清理系统 PageCache
```

```
sync && echo 3 > /proc/sys/vm/drop_caches
# 3. 模型预热
curl http://127.0.0.1:11434/api/generate -d '{"model": "qwen2.5:0.5b", "prompt":
"hi"}'
```

第 28 章 系统级深度脱水

28.1 严禁本地编译

> 🔒 **绝对不要在 4G 服务器运行 `mvn clean install`，这会诱发系统 OOM。请在本地编译 JAR 包后上传。**

28.2 一个关键的排查技巧 (如果 Java 报错)

如果你的 Spring Boot 启动时报 `Connection Refused`（连接 Milvus 失败）：

- **检查防火墙：** 阿里云安全组是否放行了 `19530` 端口（如果 Java 和 Docker 在同一台机子，通常不需要，但如果是内网访问则需注意）
- **检查 Host：** 在 `application.yml` 中，`milvus.client.host` 应该填写 `localhost` 或 `127.0.0.1`

28.3 I/O 等待检查

执行 `top` 查看 `%wa`。若长期高于 5%，说明 Swap 交换过度，建议降级使用 `0.5b` 模型。

28.4 内存隐形占用深度排查与清理

> 📊 **从最新的系统截图来看，阿里云盾（AliYunDun）已经成功卸载了（列表中已经看不到了），这是一个好的开始。**

> ⚠️ **但现在的核心问题是：** 物理内存（MiB Mem）总共约 3.5GB，已经用了 2.9GB，剩余可用（avail Mem）仅剩 515MB。

最诡异的是：`top` 列表里排名第一的 `milvus` 才用了 4.6% 的内存（约 160MB），剩下的 2.7GB 内存去哪了？这说明内存被一些"隐形"的服务或者缓存占用了。

以下是进一步"压榨"系统性能、解决卡顿的深度优化方案：

28.4.1 彻底清理 Docker 冗余（释放隐形占用）

很多时候，停止的容器、未使用的镜像和悬空卷会占用大量磁盘和内存资源。

```
# 1. 停止并移除所有已停止的容器
docker container prune -f

# 2. 清理所有悬空镜像（dangling images）
docker image prune -f

# 3. 【慎用】清理所有未被运行中容器使用的镜像（释放大量空间）
docker image prune -a -f
```

```
# 4. 清理构建缓存
docker builder prune -f

# 删除所有停止的容器、未使用的网络和挂起的镜像
docker system prune -a -f

# 5. 清理未使用的卷（确保没有重要数据在其中）
docker volume prune -f
```

28.4.2 清理系统级缓存与临时文件

Linux 内核会利用空闲内存做缓存（buff/cache），虽然理论上需要时会释放，但在内存极度紧张时，手动释放可以立竿见影。

```
# 1. 同步数据到磁盘（防止数据丢失）
sync

# 2. 清理 PageCache、dentries 和 inodes
# 1=PageCache, 2=Slab, 3=Both
# 释放网页缓存、目录项和索引
sync; echo 3 > /proc/sys/vm/drop_caches

# 检查当前日志占用大小
journalctl --disk-usage

# 将日志限制在 100MB 以内，并清理超过 1 天的日志
sudo journalctl --vacuum-time=1d
sudo journalctl --vacuum-size=100M

# 4. 清理 apt 缓存
apt-get clean && apt-get autoremove -y
```

28.4.3 揪出"隐形"内存杀手

1. 如果执行完上述操作，内存依然紧张，需要使用 `smem` 或 `ps` 命令查看更详细的内存占用（包括共享内存）。

```
# 安装 smem 工具（如果没有）
apt-get install smem -y

# 按内存占用排序查看进程（包含共享内存分摊）
smem -r -k

# 或者使用 ps 查看详细的 RSS 和 VSZ
ps aux --sort=-%mem | head -n 10
```

2. 执行这个命令，找出隐藏的内存大户（不仅仅看进程，看总和）：

```
# 查看 Docker 容器真实的内存开销
docker stats --no-stream
```

如果 `docker stats` 显示的总和很大，而 `top` 没显示，说明内存被 **Docker 的虚拟化层** 锁定了。

> 📌 **注：** 有时候 Java 进程的 Native Memory Tracking (NMT) 或者 C++ 应用的堆外内存不会完全体现在 `top` 的 `%MEM` 中，但会占用 RSS。

28.4.4 极端情况：调整 Swappiness

如果物理内存实在不够，让系统更积极地使用 Swap，避免 OOM Kill 杀掉关键进程。

```
# 临时设置（立即生效）
sysctl vm.swappiness=60

# 永久设置
echo "vm.swappiness=60" >> /etc/sysctl.conf
sysctl -p
```

> ⚠ **注意：** swappiness 过高会导致频繁的磁盘 I/O，使系统变慢，但在内存不足时能保命。

28.4.5 强制 Ollama 释放模型内存

Ollama 加载模型后，默认会一直驻留在内存中（即使你不再问答）。在 4G 内存机器上，这非常致命。

```
# 检查当前加载了哪些模型
# 如果有模型在列表里，说明它占着内存
ollama ps

# 强制重启 Ollama 服务来清空所有加载的模型
sudo systemctl restart ollama
```

建议：在你的 `VectorStoreConfig` 启动成功后，再去调用 Ollama，避免多个组件同时抢占那仅剩的 500MB。

28.4.6 检查并关闭不常用的 Linux 服务

如果你使用的是 Ubuntu/Debian，一些后台服务完全可以关掉：

```
# 1. 禁用多路径传输服务（除非你有多个硬盘路径）
sudo systemctl stop multipathd && sudo systemctl disable multipathd
```

```
# 2. 禁用自动更新服务（防止它在后台突然启动导致卡死）
sudo systemctl stop apt-daily.timer && sudo systemctl disable apt-daily.timer
sudo systemctl stop apt-daily-upgrade.timer && sudo systemctl disable apt-daily-upgrade.timer
```

28.4.7 针对 Spring Boot 的最终警告

你的截图里没看到 `java` 进程。一旦你启动 Spring Boot，它会瞬间吞掉 500MB-1GB。

- **请务必确认你的启动参数中限制了元空间（Metaspace）**，这部分内存是不计入堆内存（Xmx）的，但会占用物理内存：

```
java -Xms512m -Xmx1024m -XX:MaxMetaspaceSize=256m -jar app.jar
```

28.4.8 验证优化效果

总结建议：

1. **立即执行** `sync; echo 3 > /proc/sys/vm/drop_caches`。
2. **立即执行** `docker system prune -f`。
3. **观察** `top` 中的 `avail Mem` 是否回升到 **1000MB (1GB)** 以上。
4. 如果 `avail Mem` 依然很低，说明 **Ollama 已经把模型加载到内存了**，请执行 `sudo systemctl restart ollama`。

执行完上述步骤后，再次运行 `free -h` 和 `top`。

目标状态	说明	达标标准
avail Mem	应提升至 800MB - 1GB 以上	✅ > 800MB
top 中的 Milvus 和 Ollama	进程应能稳定运行，不再频繁波动	✅ 稳定运行

只要 `avail Mem` 维持在 1GB 以上，你的服务器操作就会非常流畅。

28.5 统启动顺序优化

> 📋 **更新重点：解决多组件启动时的资源竞争问题**

28.5.1 正确的启动顺序

```
# 第一步：启动 Milvus 全家桶（等待完全就绪）
cd /usr/milvus && docker compose up -d
# 等待 2-3 分钟，确保 Milvus 完全启动
docker compose ps  # 确认所有组件状态为 healthy

# 第二步：启动并预热 Ollama 模型
```

```
ollama pull bge-m3
ollama pull qwen2.5:1.5b
# 预热模型（防止首次请求超时）
curl http://localhost:11434/api/embeddings -d '{"model": "bge-m3", "prompt": "warmup"}'
curl http://localhost:11434/api/chat -d '{"model": "qwen2.5:1.5b", "messages": [{"role": "user", "content": "hi"}]}'

# 第三步：启动 Spring Boot 应用
java -Xms512m -Xmx600m -XX:MaxMetaspaceSize=256m -jar policy-rag-system.jar

# 第四步：验证系统健康状态
docker stats  # 观察各容器内存占用是否稳定
free -h       # 确认系统可用内存 > 200MB
```

28.5.2 启动失败应急方案

问题	应急处理	优先级
Milvus 启动失败	检查 `docker logs milvus-standalone`，确认 etcd 和 minio 先启动	🔴 高
Ollama 模型加载失败	执行 `ollama rm <模型名>` 后重新 pull	🔴 高
Java 连接 Milvus 失败	确认 `application.yml` 中 host 为 `127.0.0.1` 或容器名	🔴 高
内存不足 OOM	立即执行 `docker compose down`，清理 Docker 缓存后重启	🔴 高

28.6 内存泄漏深度排查

> 📋 ** 更新重点：解决运行一段时间后内存持续增长的问题**

28.6.1 内存泄漏常见原因

原因	症状	解决方案	检测频率
Java 堆内存泄漏	JVM 内存持续增长	限制 `-Xmx` 参数，启用 G1GC	每小时
Docker 容器内存泄漏	容器内存只增不减	设置 `deploy.resources.limits`	每小时
Ollama 模型不释放	模型常驻内存不回收	设置 `OLLAMA_KEEP_ALIVE=5m`	每 5 分钟
系统缓存累积	buff/cache 持续增长	定期执行 `drop_caches`	每日

28.6.2 内存泄漏排查工具

```
# 1. 查看 Java 进程内存详情
jstat -gc <pid> 1000 10

# 2. 查看 Docker 容器内存趋势
docker stats --no-stream
```

```
# 3. 查看 Ollama 模型占用
ollama ps

# 4. 查看系统内存详细分布
cat /proc/meminfo | grep -E "MemTotal|MemFree|MemAvailable|Buffers|Cached"
```

28.6.3 定期维护脚本（推荐每日凌晨执行）

```
#!/bin/bash
# /usr/local/bin/daily_maint.sh

echo "=== 开始每日维护 ==="
date

# 1. 重启 Ollama 释放内存碎片
echo "重启 Ollama..."
systemctl restart ollama

# 2. 清理系统缓存
echo "清理系统缓存..."
sync && echo 3 > /proc/sys/vm/drop_caches

# 3. 清理 Docker 冗余
echo "清理 Docker 冗余..."
docker container prune -f
docker image prune -f

# 4. 清理系统日志
echo "清理系统日志..."
journalctl --vacuum-time=3d

# 5. 模型预热（防止次日首次请求超时）
echo "模型预热..."
curl -s http://127.0.0.1:11434/api/generate -d '{"model": "qwen2.5:0.5b",
"prompt": "hi"}' > /dev/null

echo "=== 维护完成 ==="
date
```

添加到 crontab：

```
# 每天凌晨 3 点执行
0 3 * * * /usr/local/bin/daily_maint.sh >> /var/log/daily_maint.log 2>&1
```

28.6.4 内存监控告警（可选）

```bash
#!/bin/bash
# /usr/local/bin/memory_monitor.sh

THRESHOLD=90  # 内存使用率阈值
USED_PERCENT=$(free | grep Mem | awk '{printf("%.0f", $3/$2 * 100.0)}')

if [ $USED_PERCENT -gt $THRESHOLD ]; then
    echo "警告：内存使用率超过 ${THRESHOLD}% (当前：${USED_PERCENT}%)" | tee -a
/var/log/memory_alert.log
    # 可选：发送告警邮件或短信
    # mail -s "内存告警" admin@example.com < /var/log/memory_alert.log
fi
```

第七部分：生产部署与运维

第 29 章 容器化部署方案

29.1 Docker Compose 编排模板

```yaml
services:
   etcd:
      image: quay.io/coreos/etcd:v3.5.5
      container_name: milvus-etcd
      command: etcd -advertise-client-urls=http://127.0.0.1:2379 -listen-client-
urls=http://0.0.0.0:2379 --data-dir=/etcd

   minio:
      image: minio/minio:RELEASE.2023-03-20T20-16-18Z
      container_name: milvus-minio
      environment:
         MINIO_ACCESS_KEY: minioadmin
         MINIO_SECRET_KEY: minioadmin
      command: minio server /export --console-address ":9001"

   milvus:
      image: milvusdb/milvus:v2.3.15
      container_name: milvus-standalone
      ports:
         - "19530:19530"
      depends_on: ["etcd", "minio"]
      volumes:
         - ./volumes/milvus:/var/lib/milvus  # 必须配置持久化

   java-app:
      build: ./java-service
      container_name: my-java-app
      ports:
         - "8080:8080"
```

```
        environment:
          - MILVUS_HOST=milvus-standalone  # 关键：使用服务名通信
        depends_on: ["milvus"]
```

29.2 部署关键要点

要点	说明	优先级	常见错误
服务发现	在 Docker 网络中，Java 连接 Milvus 的 Host 必须写服务名 `milvus-standalone`，严禁写 `localhost`	🔴 高	写 localhost 导致连接失败
安全组开放 (阿里云后台)	19530：Milvus gRPC 接口9000/9001：Portainer 或 MinIO 控制台8080：Java API 接口	🔴 高	忘记开放端口
数据持久化	务必在 Compose 文件中配置 `volumes`，否则 Milvus 里的向量数据和数据库内容会在容器删除后丢失	🔴 高	未配置 volumes

29.3 实战部署三步走

第一步：启动并预热模型 (Ollama)

```
# 预热 Embedding 模型 (BGE-M3)
curl http://localhost:11434/api/embeddings -d '{
    "model": "bge-m3",
    "prompt": "warmup"
}'

# 预热 Chat 模型 (Qwen2.5-1.5B)
curl http://localhost:11434/api/chat -d '{
    "model": "qwen2.5:1.5b",
    "messages": [{"role": "user", "content": "hi"}]
}'
```

第二步：启动 Spring Boot 应用

```
# 进入你的 Jar 包目录
java -Xms512m -Xmx600m -jar your-rag-project.jar
```

第三步：最终内存动态监控

```
watch -n 1 free -h
```

第 30 章 故障排查与常见问题

30.1 Ollama 常见问题

问题	原因	解决方案	是否可忽略
警告 "CPU-only mode"	通常是驱动未识别	检查 `nvidia-smi` 是否正常，并重启 ollama 服务	❌ 否 (GPU 环境)
`nvidia-smi` 未找到	非 GPU 实例	可忽略	✅ 是 (CPU 环境)
`aplay command not found`	阿里云精简镜像缺少组件	`apt update && apt install alsa-utils -y`	❌ 否

30.2 Milvus 常见问题

问题	原因	解决方案	优先级
连接超时	Docker 容器端口 19530 被云服务器安全组拦截	检查阿里云安全组配置	🔴 高
Connection Refused (19530)	服务未启动或安全组未开放	检查 Milvus 容器是否 OOM（查看 `docker stats`）；如果是容器间通信，Host 请使用容器名 `milvus-standalone` 而非 `localhost`	🔴 高
内存不足 (OOM)	Milvus 的数据是预加载到内存的	如果内存不足 16GB，建议限制 Milvus 的内存配额	🔴 高

30.3 Docker 常见错误

错误	对策	优先级
`i/o timeout`	镜像源失效。更换 `daemon.json` 里的地址或使用阿里云个人镜像	🔴 高
`the attribute version is obsolete`	YAML 警告。删除 `docker-compose.yml` 第一行的 `version: '3.x'` 即可	⚪ 低
`Job for docker.service failed`	检查 `daemon.json` 格式（逗号、引号、大括号是否配对）	🔴 高
WSL/BIOS 报错	Windows 环境需执行 `wsl --update` 或在 BIOS 开启虚拟化技术	🔴 高

30.4 Spring AI 常见问题

问题	原因	解决方案	优先级
`RestClientAutoConfiguration not present`	Spring AI 1.0.0+ 依赖 RestClient	确保 Spring Boot 版本 ≥ 3.2.0	🔴 高
`TypeTag :: UNKNOWN`	JDK 25/Lombok 冲突	降级至 JDK 21 并在 `maven-compiler-plugin` 显式配置 Lombok 路径	🔴 高
`collection not found`	M6 版本自动配置不稳定	使用 Java 配置类强制接管，显式设置 `.initializeSchema(true)`	🔴 高
维度不匹配	`embedding-dimension` 与模型输出维度不符	更换模型必须删除旧的 Collection，确保维度一致	🔴 高

30.5 综合排查流程

1. **检查网络：** 本地执行 `telnet 8.140.xxx 19530`
    - 若 Timeout → 安全组没开
    - 若 Refused → 服务没起
2. **检查依赖：** 查看日志 `docker logs milvus-standalone`
    - 重点看 etcd 和 MinIO 是否连通
3. **重启大法：** 执行 `docker compose down` 彻底清理后，再 `docker compose up -d`
4. **内存检查：** 执行 `docker stats` 和 `free -h`
    - 确保各组件内存占用在预期范围内

第 31 章 自动化运维与监控

31.1 运行健康指标 (Checklist)

检查项	标准	检查频率	告警阈值
内存监控	执行 `free -m`，`avail` 需保持在 200MB 以上	每分钟	< 100MB
内网对齐	`application.yml` 中的 `host` 必须使用 `127.0.0.1` 以绕过公网防火墙并降低延迟	部署时	-
维度一致性	`BGE-M3` 必须对应 `1024` 维度，否则 `SimilaritySearch` 会报错	部署时	-
I/O 等待	执行 `top` 查看 `%wa`。若长期高于 5%，说明 Swap 交换过度	每小时	> 5%

31.2 日志管理

使用 `logback-spring.xml` 将 ERROR 日志与常规日志分离，方便快速定位 RAG 检索失败的原因。

```
logging:
  file:
    name: logs/rag.log
  logback:
    rollingpolicy:
      file-name-pattern: logs/rag-%d{yyyy-MM-dd}.%i.log
      max-file-size: 100MB
```

31.3 数据备份

生产环境务必定期备份 `volumes` 挂载目录，那是你向量数据库的命脉。

```
# 备份 Milvus 数据
tar -czf milvus-backup-$(date +%Y%m%d).tar.gz ./volumes/milvus
```

第 32 章 安全与性能最佳实践

32.1 安全性建议

建议	说明	优先级	实施难度
Portainer 密码	设置完密码后，建议在阿里云安全组限制访问 9000 端口的来源 IP	● 高	☆
网络隔离	生产环境下，Etcd、MinIO、Milvus 的内部端口（除 19530 外）尽量不要暴露给公网	● 高	☆☆
跨域配置	生产环境应限制 `@CrossOrigin` 的允许来源，不要使用 `*`	○ 中	☆

32.2 性能最佳实践

优化项	建议	预期提升	实施难度
Milvus 索引类型	设为 HNSW	50-70%	☆☆
Ollama 模型常驻	生产环境建议设置 `OLLAMA_KEEP_ALIVE=-1` 防止模型卸载导致下次请求卡顿	30-50%	☆
模型量化	使用 `q4` 或 `q5` 量化版本	30-50%	☆

优化项	建议	预期提升	实施难度
分批处理	文档入库时每批 32 条最稳健	稳定性提升	☆

32.3 进阶建议

> 💡 **专家建议：** 后期若业务增加，可考虑将推理 API 托管至 **阿里云 DashScope (通义千问 API)**。届时服务器仅运行 Spring Boot + Milvus，内存占用会降至 1.5G 以下，系统将变得飞速且极其稳定！

第 33 章 关键遗漏点补充

33.1 网络与安全组配置详解

阿里云安全组完整配置：

端口	协议	授权对象	说明	优先级
22	TCP	0.0.0.0/0	SSH 远程连接	🔴 必须
11434	TCP	0.0.0.0/0	Ollama API	🔴 必须
19530	TCP	0.0.0.0/0	Milvus gRPC	🔴 必须
9000	TCP	你的 IP/32	Portainer (建议限制 IP)	⚪ 推荐
9001	TCP	你的 IP/32	MinIO 控制台	⚪ 推荐
8080	TCP	0.0.0.0/0	Java API	🔴 必须

⚠ **安全提醒：** 生产环境建议将 9000/9001 端口限制为仅允许你的办公 IP 访问。

33.2 模型量化详解

量化等级对比：

量化级别	精度损失	内存占用	推理速度	推荐场景
Q8_0	<1%	高	快	生产环境
Q5_K_M	~2%	中	很快	推荐
Q4_K_M	~3%	低	极快	低配机器
Q3_K_S	~5%	极低	最快	极限低配

查看模型量化信息：

```
ollama list
# 输出示例：
# NAME              ID            SIZE      MODIFIED
# qwen2.5:1.5b     65ec06548149  986 MB    2 days ago
```

33.3 Prompt 工程优化

基础 Prompt 模板：

```
基于以下政策上下文回答问题，如果上下文中没有相关信息，请明确告知用户。

政策上下文：
{context}

问题：{query}

回答：
```

进阶 Prompt 模板（带引用）：

```
你是一名政务政策问答助手。请基于以下检索到的政策片段回答问题。

要求：
1. 仅根据提供的上下文回答，不要编造信息
2. 如果上下文不足以回答问题，请明确告知用户
3. 回答末尾请注明引用的政策文件名称

政策上下文：
{context}

问题：{query}

回答：
```

33.4 文档上传最佳实践

建议	说明	优先级
文件格式	优先使用 PDF/Word，避免扫描件（需 OCR）	● 高
文件大小	单文件建议 < 50MB，大文件拆分上传	○ 中
命名规范	使用有意义的文件名，便于追溯来源	○ 中
分批上传	大量文档时分批上传，避免一次性 OOM	● 高

33.5 性能基准测试

推荐测试工具：

```
# 测试 Ollama 推理速度
ollama run qwen2.5:1.5b "请用 100 字介绍 RAG 技术"

# 测试 Milvus 检索延迟
# 在 Java 代码中添加计时
long start = System.currentTimeMillis();
vectorStore.similaritySearch(query);
long end = System.currentTimeMillis();
log.info("检索耗时：{}ms", end - start);

# 测试端到端响应时间
time curl "http://localhost:8080/api/chat?query=什么是 RAG"

# 使用 ab 进行压力测试
ab -n 1000 -c 10 http://localhost:8080/api/chat?query=测试问题

# 使用 wrk 进行高级压力测试
wrk -t12 -c400 -d30s http://localhost:8080/api/chat?query=测试问题
```

性能目标：

指标	目标值	说明	测量工具
QPS	> 10	单实例并发处理能力	ab/wrk
P99 延迟	< 5s	99% 请求的响应时间	ab/wrk
错误率	< 1%	请求失败比例	ab/wrk

- 检索延迟：< 100ms
- 首字延迟 (TTFT)：< 2s
- 完整响应：< 10s

33.6 文档预处理最佳实践

PDF 解析注意事项：

问题	解决方案
表格解析错乱	使用 Apache Tika 1.28+ 版本
扫描版 PDF 无法解析	需先 OCR 处理（Tesseract）
多栏排版错乱	使用专门的 PDF 解析库（如 PyMuPDF）
图片中的文字	需要 OCR 提取后单独处理

文档切片优化：

```
// 推荐配置：按段落智能切片
TextSplitter splitter = new TokenTextSplitter(
                500,    // 每片 token 数
```

```
                100,    // 重叠 token 数（保证语义连贯）
                10,     // 最小片大小
                10000,  // 最大片大小
                true    // 保持段落完整
        );
```

33.7 监控与告警配置

Prometheus + Grafana 监控方案：

```yaml
# docker-compose-monitoring.yml
services:
   prometheus:
      image: prom/prometheus
      volumes:
         - ./prometheus.yml:/etc/prometheus/prometheus.yml
      ports:
         - "9090:9090"

   grafana:
      image: grafana/grafana
      ports:
         - "3000:3000"
      environment:
         - GF_SECURITY_ADMIN_PASSWORD=admin123
```

关键监控指标：

- JVM 内存使用率
- Milvus 查询延迟
- Ollama 推理速度
- 系统内存/Swap 使用率
- 磁盘空间使用率

33.8 灾难恢复计划

数据恢复流程：

```bash
# 1. 停止所有服务
docker compose down

# 2. 恢复 Milvus 数据
tar -xzf milvus-backup-20240101.tar.gz -C ./

# 3. 重启服务
docker compose up -d

# 4. 验证数据完整性
curl http://localhost:19530/api/collections
```

备份策略：

- 每日增量备份
- 每周全量备份
- 备份保留 30 天

第 34 章 企业级 RAG 安全与合规

34.1 数据安全五层防护体系

层级	防护措施	实现方式
网络层	隔离内外网	安全组、VPC、防火墙
传输层	加密通信	HTTPS/TLS 1.3
应用层	身份认证与授权	JWT、OAuth2、RBAC
数据层	敏感数据脱敏	字段加密、数据掩码
审计层	操作日志记录	完整审计日志、异常告警

措施	说明	实施难度	优先级
传输加密	生产环境使用 HTTPS，配置 SSL 证书	☆☆☆	● 高
数据脱敏	敏感信息（如身份证、手机号）在入库前脱敏	☆☆	● 高
访问控制	实现用户认证与授权，限制 API 访问	☆☆☆	● 高
审计日志	记录所有问答操作，便于追溯	☆☆	○ 中

34.2 合规要求

要求	实现方式	法规依据	优先级
数据本地化	确保所有数据存储在境内服务器	《网络安全法》	● 高
隐私保护	遵循《个人信息保护法》，用户数据加密存储	《个人信息保护法》	● 高
内容审核	对生成内容进行敏感词过滤	《生成式 AI 管理办法》	● 高
备份策略	定期备份向量数据库，防止数据丢失	《数据安全法》	○ 中

34.3 阿里云安全组完整配置

入方向规则：

优先级	端口	授权对象	协议	说明
1	22	办公 IP/32	TCP	SSH 管理
2	443	0.0.0.0/0	TCP	HTTPS

优先级	端口	授权对象	协议	说明
3	8080	0.0.0.0/0	TCP	API 接口
4	11434	内网网段	TCP	Ollama
5	19530	内网网段	TCP	Milvus
6	9000	办公 IP/32	TCP	Portainer
7	9001	办公 IP/32	TCP	MinIO

> ⚠ **生产环境严禁将 11434、19530 端口对公网开放！**

34.4 数据出境合规性检查

涉及场景： 使用境外 AI API（如 OpenAI、Anthropic）

检查项	要求	解决方案
数据分类	识别敏感数据	建立数据分类分级制度
用户同意	获取明确授权	隐私政策中明确说明
数据脱敏	去除个人标识	本地预处理后再调用 API
境内存储	原始数据不出境	向量数据本地存储
审计记录	保留访问日志	日志留存≥6 个月

> 💡 **推荐方案：** 使用本地部署的 Ollama + 通义千问 API，避免数据出境风险。

34.5 敏感信息过滤

```java
@Component
public class SensitiveDataFilter {

    private static final List<Pattern> SENSITIVE_PATTERNS = List.of(
            Pattern.compile("\\d{18}"),  // 身份证号
            Pattern.compile("1[3-9]\\d{9}"),  // 手机号
            Pattern.compile("[\\w.-]+@[\\w.-]+\\.[\\w]+"),  // 邮箱
            Pattern.compile("\\d{4}[- ]?\\d{4}[- ]?\\d{4}[- ]?\\d{4}")  // 银行卡号
    );

    // 在入库前过滤敏感信息
    public String filterSensitiveData(String text) {
        String filtered = text;
        for (Pattern pattern : SENSITIVE_PATTERNS) {
            filtered = pattern.matcher(filtered).replaceAll("[已脱敏]");
        }
        return filtered;
    }
}
```

第 35 章 故障排查决策树 (增强版)

35.1 三级风险管控体系

风险等级	颜色	响应时间	处理流程
红色警报	🔴	5 分钟内	系统完全不可用，立即启动应急预案
黄色预警	🟡	30 分钟内	性能下降或部分功能异常，安排紧急修复
绿色正常	🟢	日常处理	系统正常运行，定期巡检

35.2 红色警报：系统完全不可用

症状： 所有 API 请求返回 500 或超时

排查流程：

1. 检查服务器状态
    - ssh 登录服务器
    - 执行 top 查看 CPU/内存
    - 执行 free -h 检查内存是否耗尽
2. 检查 Docker 容器
    - docker ps -a 查看容器状态
    - docker stats 查看资源占用
    - 若容器 Exited，执行 docker logs <容器名> 查看错误
3. 检查关键服务
    - systemctl status ollama
    - docker compose ps (Milvus 相关)
    - 若 Ollama 停止，执行 systemctl restart ollama
4. 紧急恢复
    - 释放内存：sync && echo 3 > /proc/sys/vm/drop_caches
    - 重启服务：docker compose restart
    - 验证：curl http://localhost:8080/api/health

35.3 黄色预警：性能下降

症状： 响应时间>5s 或检索准确率下降

排查流程：

1. 检查检索延迟
    - 查看应用日志中的检索耗时
    - 若>500ms，检查 Milvus 负载

2. 检查模型推理速度
    - curl -w "@curl-format.txt" http://localhost:11434/api/generate
    - 若 tokens/s < 10，检查 Ollama 是否使用 GPU
3. 检查内存压力
    - free -h 查看 Swap 使用率
    - 若 Swap>50%，考虑升级内存或降级模型
4. 优化措施
    - 清理 Milvus 旧数据：ATTU 客户端删除过期 Collection
    - 限制 Ollama 并发：OLLAMA_NUM_PARALLEL=1
    - 启用查询缓存：实现 Redis 缓存层

系统故障 │ ├─ 无法访问服务 │ ├─ 检查安全组端口是否开放 │ ├─ 检查服务是否运行 (docker ps / systemctl status) │ └─ 检查防火墙配置 │ ├─ 响应超时 │ ├─ 检查内存使用 (free -h) │ ├─ 检查 CPU 负载 (top) │ ├─ 检查 I/O 等待 (%wa) │ └─ 检查 Swap 使用率 │ ├─ 检索结果不准确 │ ├─ 检查 Embedding 模型维度是否匹配 │ ├─ 检查 Milvus 索引类型 │ ├─ 调整相似度阈值 │ └─ 优化文档分块策略 │ └─ 内存溢出 (OOM) ├─ 限制各组件内存配额 ├─ 开启 Swap 虚拟内存 ├─ 降级模型 (7B → 1.5B → 0.5B) └─ 清理系统缓存和 Docker 冗余

35.4 常见错误代码速查

错误代码	含义	解决方案
Connection Refused 19530	Milvus 未启动或端口被防火墙拦截	检查容器状态和安全组
collection not found	Collection 不存在或名称配置错误	检查 initializeSchema(true)
dimension mismatch	向量维度与 Collection 定义不匹配	删除 Collection 后重建
OOM Killer	内存不足导致进程被杀	增加 Swap 或限制 JVM 堆内存
context_length_exceeded	输入超过模型上下文限制	减少 Top-K 或切片大小
rate_limit_exceeded	API 调用频率超限	增加重试机制和限流

第 36 章 性能基准测试与优化目标

36.1 性能测试工具

```
# 1. 测试 Ollama 推理速度
ollama run qwen2.5:1.5b "请用 100 字介绍 RAG 技术"

# 2. 测试 Milvus 检索延迟（使用 milvus-cli）
pip install milvus-cli
milvus-cli connection set --alias default --host localhost --port 19530
milvus-cli collection describe --collection-name policy_docs

# 3. 测试端到端响应时间
time curl "http://localhost:8080/api/chat?query=什么是 RAG"
```

```
# 4. 压力测试（使用 wrk）
wrk -t4 -c100 -d30s http://localhost:8080/api/chat?query=test
```

测试脚本

```
#!/bin/bash
# performance_test.sh

echo "=== RAG 系统性能基准测试 ==="

# 测试参数
TOTAL_REQUESTS=100
CONCURRENT_USERS=5
TEST_QUERY="人才补贴如何申请？"

echo "开始测试..."
start_time=$(date +%s)

for i in $(seq 1 $TOTAL_REQUESTS); do
    curl -s "http://localhost:8080/api/chat?query=${TEST_QUERY}" > /dev/null &

    # 控制并发数
    if [ $((i % CONCURRENT_USERS)) -eq 0 ]; then
        wait
    fi
done

wait
end_time=$(date +%s)

duration=$((end_time - start_time))
qps=$((TOTAL_REQUESTS / duration))

echo "测试完成！"
echo "总请求数：$TOTAL_REQUESTS"
echo "耗时：${duration}秒"
echo "QPS: $qps"
```

36.2 性能目标基准（2026 年行业标准）

指标	低配服务器 (2C4G)	标准服务器 (8C32G)	GPU 服务器 (A10)
检索延迟 (P95)	<100ms	<50ms	<20ms
首字延迟 (TTFT)	<3s	<2s	<1s
完整响应时间	<15s	<10s	<5s
并发 QPS	5-10	20-50	100+

指标	低配服务器 (2C4G)	标准服务器 (8C32G)	GPU 服务器 (A10)
检索准确率	>85%	>90%	>92%
系统可用性	99%	99.5%	99.9%

36.3 性能优化优先级

优化优先级排序（按投入产出比）：

1. ☆☆☆☆☆ 启用 Swap 虚拟内存（成本 0，效果显著）
2. ☆☆☆☆☆ 选择合适的 Embedding 模型（bge-m3 vs bge-small）
3. ☆☆☆☆ 调整分块策略（500 token + 20% 重叠）
4. ☆☆☆☆ 优化 Milvus 索引参数（HNSW M=16）
5. ☆☆☆ 限制 Ollama 并发（OLLAMA_NUM_PARALLEL=1）
6. ☆☆☆ 启用查询缓存（Redis 缓存热点查询）
7. ☆☆ 升级硬件（GPU 或增加内存）
8. ☆ 使用云端 API（DashScope 替代本地 Ollama）

36.4 优化目标对照表

硬件配置	QPS 目标	P99 延迟	内存占用	适用场景
2 核 4G (CPU)	2-5	< 10s	< 3.5G	个人实验
4 核 8G (CPU)	5-10	< 5s	< 7G	小型生产
8 核 16G + GPU	10-20	< 3s	< 14G	中型生产
16 核 32G + GPU	20-50	< 2s	< 28G	大型生产

第 37 章 监控告警体系

37.1 Prometheus + Grafana 监控配置

```yaml
# docker-compose-monitoring.yml
services:
   prometheus:
      image: prom/prometheus:v2.45.0
      container_name: prometheus
      volumes:
         - ./prometheus.yml:/etc/prometheus/prometheus.yml
         - prometheus_data:/prometheus
      ports:
         - "9090:9090"
      command:
         - '--config.file=/etc/prometheus/prometheus.yml'
         - '--storage.tsdb.path=/prometheus'
         - '--storage.tsdb.retention.time=15d'

   grafana:
```

```yaml
        image: grafana/grafana:10.0.0
        container_name: grafana
        volumes:
           - grafana_data:/var/lib/grafana
        ports:
           - "3000:3000"
        environment:
           - GF_SECURITY_ADMIN_PASSWORD=admin123
           - GF_USERS_ALLOW_SIGN_UP=false

     node-exporter:
        image: prom/node-exporter:v1.6.0
        container_name: node-exporter
        ports:
           - "9100:9100"
        volumes:
           - /proc:/host/proc:ro
           - /sys:/host/sys:ro
           - /:/rootfs:ro
        command:
           - '--path.procfs=/host/proc'
           - '--path.sysfs=/host/sys'

  volumes:
     prometheus_data:
     grafana_data:
```

37.2 关键监控指标

指标类别	具体指标	告警阈值	告警级别
系统资源	CPU 使用率	>80% 持续 5 分钟	⚪ 警告
	内存使用率	>90% 持续 5 分钟	⚫ 严重
	磁盘使用率	>85%	⚪ 警告
Milvus	检索延迟 (P95)	>200ms	⚪ 警告
	查询 QPS	>100	⚪ 警告
	内存占用	>1.5GB	⚫ 严重
Ollama	推理速度 (tokens/s)	<10	⚪ 警告
	模型加载时间	>30s	⚪ 警告
应用层	API 错误率	>5%	⚫ 严重
	平均响应时间	>5s	⚪ 警告
	活跃连接数	>500	⚪ 警告

37.3 告警通知配置

```yaml
# alertmanager.yml
global:
   smtp_smarthost: 'smtp.example.com:587'
   smtp_from: 'alert@example.com'

route:
   group_by: ['alertname']
   group_wait: 30s
   group_interval: 5m
   repeat_interval: 4h
   receiver: 'email-notifications'

receivers:
   - name: 'email-notifications'
     email_configs:
        - to: 'devops@example.com'
          send_resolved: true

inhibit_rules:
   - source_match:
        severity: 'critical'
     target_match:
        severity: 'warning'
     equal: ['alertname', 'instance']
```

第 27.12 章 灾难恢复与备份策略

27.12.1 数据备份方案

```bash
#!/bin/bash
# daily_backup.sh - 每日自动备份脚本

BACKUP_DIR="/backup/rag-system"
DATE=$(date +%Y%m%d_%H%M%S)
RETENTION_DAYS=30

# 1. 备份 Milvus 数据
echo "备份 Milvus 数据..."
tar -czf ${BACKUP_DIR}/milvus_${DATE}.tar.gz ./volumes/milvus

# 2. 备份 MySQL/PostgreSQL（如有）
echo "备份数据库..."
mysqldump -u root -p${DB_PASSWORD} rag_db > ${BACKUP_DIR}/mysql_${DATE}.sql

# 3. 备份应用配置
echo "备份配置文件..."
cp -r ./config ${BACKUP_DIR}/config_${DATE}

# 4. 清理旧备份
```

```
echo "清理${RETENTION_DAYS}天前的备份..."
find ${BACKUP_DIR} -name "*.tar.gz" -mtime +${RETENTION_DAYS} -delete
find ${BACKUP_DIR} -name "*.sql" -mtime +${RETENTION_DAYS} -delete

echo "备份完成：${DATE}"
```

27.12.2 备份策略建议

数据类型	备份频率	保留周期	存储位置
Milvus 向量数据	每日增量 + 每周全量	30 天	对象存储 (OSS/S3)
应用配置文件	每次变更后	永久	Git 仓库
日志文件	每日轮转	90 天	本地 + 远程
模型文件	一次性	永久	本地 + 备份盘

27.12.3 灾难恢复流程

```
恢复流程（目标 RTO<4 小时，RPO<24 小时）：

1. 评估损失 (15 分钟)
   └ 确定故障范围
   └ 选择恢复点（最近备份）

2. 准备环境 (30 分钟)
   └ 准备新服务器或修复原服务器
   └ 安装 Docker、Ollama 等基础组件

3. 恢复数据 (2 小时)
   └ 解压 Milvus 备份：tar -xzf milvus_20260312.tar.gz
   └ 恢复数据库：mysql -u root -p < mysql_20260312.sql
   └ 恢复配置文件

4. 启动服务 (30 分钟)
   └ docker compose up -d
   └ systemctl start ollama
   └ 启动 Spring Boot 应用

5. 验证功能 (15 分钟)
   └ 执行健康检查
   └ 运行测试用例
   └ 确认业务恢复正常

6. 事后分析 (1 小时)
   └ 编写事故报告
   └ 制定预防措施
```

附录：命令速查与配置模板

附录 A：完整配置文件模板

A.1 docker-compose.yml (完整版)

```yaml
version: '3.8'

services:
   etcd:
      container_name: milvus-etcd
      image: quay.io/coreos/etcd:v3.5.5
      environment:
         - ETCD_AUTO_COMPACTION_MODE=revision
         - ETCD_AUTO_COMPACTION_RETENTION=1000
         - ETCD_QUOTA_BACKEND_BYTES=4294967296
         - ETCD_SNAPSHOT_COUNT=50000
      volumes:
         - ./volumes/etcd:/etcd
      command: etcd -advertise-client-urls=http://127.0.0.1:2379 -listen-client-
urls http://0.0.0.0:2379 --data-dir /etcd
      healthcheck:
         test: ["CMD", "etcdctl", "endpoint", "health"]
         interval: 30s
         timeout: 20s
         retries: 3
      deploy:
         resources:
            limits:
               memory: 256M

   minio:
      container_name: milvus-minio
      image: docker.m.daocloud.io/minio/minio:RELEASE.2023-03-20T20-16-18Z
      environment:
         MINIO_ACCESS_KEY: minioadmin
         MINIO_SECRET_KEY: minioadmin
      ports:
         - "9001:9001"
         - "9000:9000"
      volumes:
         - ./volumes/minio:/minio_data
      command: minio server /minio_data --console-address ":9001"
      healthcheck:
         test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]
         interval: 30s
         timeout: 20s
         retries: 3
      deploy:
         resources:
            limits:
```

```yaml
                memory: 512M

    standalone:
        container_name: milvus-standalone
        image: docker.m.daocloud.io/milvusdb/milvus:v2.4.0
        command: ["milvus", "run", "standalone"]
        security_opt:
            - seccomp:unconfined
        environment:
            ETCD_ENDPOINTS: etcd:2379
            MINIO_ADDRESS: minio:9000
            CACHE_SIZE: 512M
        volumes:
            - ./volumes/milvus:/var/lib/milvus
        healthcheck:
            test: ["CMD", "curl", "-f", "http://localhost:9091/healthz"]
            interval: 30s
            start_period: 90s
            timeout: 20s
            retries: 3
        ports:
            - "19530:19530"
            - "9091:9091"
        depends_on:
            etcd:
                condition: service_healthy
            minio:
                condition: service_healthy
        deploy:
            resources:
                limits:
                    memory: 1024M

networks:
    default:
        name: milvus
```

A.2 application.yml (完整版)

```yaml
spring:
    application:
        name: policy-rag-system
    ai:
        ollama:
            base-url: http://127.0.0.1:11434
            chat:
                model: qwen2.5:1.5b
                options:
                    temperature: 0.3
            embedding:
                model: bge-m3
```

```yaml
      vectorstore:
         milvus:
            client:
               host: 127.0.0.1
               port: 19530
            collection-name: policy_docs
            embedding-dimension: 1024
            index-type: HNSW
   mvc:
      async:
         request-timeout: 300000

server:
   port: 8080

logging:
   level:
      com.ailearn.rag: DEBUG
      org.springframework.ai: DEBUG
   file:
      name: logs/rag.log
   logback:
      rollingpolicy:
         file-name-pattern: logs/rag-%d{yyyy-MM-dd}.%i.log
         max-file-size: 100MB
         max-history: 30
```

附录 B：完整 Java 代码示例

B.1 PolicyRagApplication.java

```java
package com.ailearn.rag;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class PolicyRagApplication {
   public static void main(String[] args) {
      SpringApplication.run(PolicyRagApplication.class, args);
   }
}
```

B.2 VectorStoreConfig.java

```java
package com.ailearn.rag.config;

import io.milvus.client.MilvusServiceClient;
```

```java
import org.springframework.ai.embedding.EmbeddingModel;
import org.springframework.ai.vectorstore.MilvusVectorStore;
import org.springframework.ai.vectorstore.MilvusVectorStoreProperties;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Primary;
import org.springframework.boot.context.properties.EnableConfigurationProperties;

@Configuration
@EnableConfigurationProperties(MilvusVectorStoreProperties.class)
public class VectorStoreConfig {

   @Bean
   @Primary
   public MilvusVectorStore vectorStore(MilvusServiceClient client,
           EmbeddingModel model,
           MilvusVectorStoreProperties properties) {
      return MilvusVectorStore.builder(client, model)
              .collectionName(properties.getCollectionName())
              .embeddingDimension(properties.getEmbeddingDimension())
              .initializeSchema(true)
              .build();
   }
}
```

B.3 CorsConfig.java

```java
package com.ailearn.rag.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class CorsConfig implements WebMvcConfigurer {
   @Override
   public void addCorsMappings(CorsRegistry registry) {
      registry.addMapping("/**")
              .allowedOrigins("*")
              .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
              .allowedHeaders("*")
              .maxAge(3600);
   }
}
```

B.4 ChatController.java

```java
package com.ailearn.rag.controller;
```

```java
import com.ailearn.rag.service.RagService;
import com.ailearn.rag.service.IngestionService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.MediaType;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;
import reactor.core.publisher.Flux;

import java.io.IOException;

@RestController
@RequestMapping("/api")
@CrossOrigin(origins = "*")
@RequiredArgsConstructor
public class ChatController {

    private final RagService ragService;
    private final IngestionService ingestionService;

    @GetMapping(value = "/chat", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    public Flux<String> chat(@RequestParam String query) {
        return ragService.streamAnswer(query);
    }

    @PostMapping("/upload")
    public String upload(@RequestParam("file") MultipartFile file) throws
IOException {
        ingestionService.processDocument(file);
        return "上传并解析成功：" + file.getOriginalFilename();
    }
}
```

B.5 RagService.java

```java
package com.ailearn.rag.service;

import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.document.Document;
import org.springframework.ai.vectorstore.SearchRequest;
import org.springframework.ai.vectorstore.VectorStore;
import org.springframework.stereotype.Service;
import reactor.core.publisher.Flux;

import java.util.List;
import java.util.stream.Collectors;

@Service
@RequiredArgsConstructor
public class RagService {

    private final ChatClient chatClient;
```

```
    private final VectorStore vectorStore;

    public RagService(ChatClient.Builder chatClientBuilder, VectorStore
vectorStore) {
        this.chatClient = chatClientBuilder.build();
        this.vectorStore = vectorStore;
    }

    public Flux<String> streamAnswer(String query) {
        List<Document> docs = vectorStore.similaritySearch(
                SearchRequest.builder()
                        .query(query)
                        .topK(3)
                        .similarityThreshold(0.6)
                        .build()
        );

        String context = docs.stream()
                .map(Document::getText)
                .collect(Collectors.joining("\n\n"));

        String refs = docs.stream()
                .map(d -> (String) d.getMetadata().getOrDefault("filename", "未知来
源"))
                .distinct()
                .collect(Collectors.joining(",  "));

        String prompt = "基于以下政策上下文回答问题：\n" + context + "\n\n问题：" +
query;

        return chatClient.prompt()
                .user(prompt)
                .stream()
                .content()
                .concatWith(Flux.just("\n\n---\n📚 来源：" + refs));
    }
}
```

B.6 IngestionService.java

```
package com.ailearn.rag.service;

import org.springframework.ai.document.Document;
import org.springframework.ai.reader.tika.TikaDocumentReader;
import org.springframework.ai.transformer.splitter.TokenTextSplitter;
import org.springframework.ai.vectorstore.VectorStore;
import org.springframework.core.io.InputStreamResource;
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;

import java.io.IOException;
```

```java
import java.util.List;

@Service
@RequiredArgsConstructor
public class IngestionService {
   private final VectorStore vectorStore;

   public void processDocument(MultipartFile file) throws IOException {
      TikaDocumentReader loader = new TikaDocumentReader(
              new InputStreamResource(file.getInputStream())
      );

      TokenTextSplitter splitter = new TokenTextSplitter(500, 100, 10, 10000,
true);
      List<Document> splitDocuments = splitter.apply(loader.get());

      splitDocuments.forEach(doc ->
              doc.getMetadata().put("filename", file.getOriginalFilename())
      );

      int batchSize = 32;
      for (int i = 0; i < splitDocuments.size(); i += batchSize) {
         vectorStore.add(splitDocuments.subList(i, Math.min(i + batchSize,
splitDocuments.size())));
      }
   }
}
```

附录 C：完整前端代码示例

C.1 package.json

```json
{
   "name": "rag-frontend",
   "version": "1.0.0",
   "scripts": {
      "dev": "vite",
      "build": "vite build",
      "preview": "vite preview"
   },
   "dependencies": {
      "vue": "^3.4.0",
      "@microsoft/fetch-event-source": "^2.0.1",
      "markdown-it": "^14.0.0"
   },
   "devDependencies": {
      "@vitejs/plugin-vue": "^5.0.0",
      "vite": "^5.0.0"
   }
}
```

C.2 ChatBox.vue (完整组件)

```
<template>
   <div class="chat-container">
      <div class="messages" ref="msgBox">
         <div v-for="(msg, index) in messages" :key="index"
              :class="['message', msg.role]">
            <div class="role">{{ msg.role === 'user' ? '👤 用户' : '🤖 助手' }}
</div>
            <div class="content" v-html="renderMarkdown(msg.content)"></div>
            <div v-if="msg.sources" class="sources">
               <strong>📚 来源：</strong>{{ msg.sources }}
            </div>
         </div>
         <div v-if="loading" class="loading">思考中...</div>
      </div>

      <div class="input-area">
         <input
                 v-model="userInput"
                 @keyup.enter="sendMessage"
                 placeholder="请输入政策问题..."
                 :disabled="loading"
         />
         <button @click="sendMessage" :disabled="loading">
            {{ loading ? '发送中...' : '发送' }}
         </button>
      </div>
   </div>
</template>

<script setup lang="ts">
   import { ref } from 'vue'
   import MarkdownIt from 'markdown-it'

   const md = new MarkdownIt()
   const userInput = ref('')
   const messages = ref<{ role: string; content: string; sources?: string }[]>([])
   const loading = ref(false)
   const msgBox = ref<HTMLElement | null>(null)

   const renderMarkdown = (text: string) => {
      return md.render(text)
   }

   const scrollToBottom = () => {
      if (msgBox.value) {
         msgBox.value.scrollTop = msgBox.value.scrollHeight
      }
   }
```

```
   const sendMessage = async () => {
      if (!userInput.value.trim()) return

      const query = userInput.value
      messages.value.push({ role: 'user', content: query })
      userInput.value = ''
      loading.value = true
      scrollToBottom()

      const assistantMsg = { role: 'assistant', content: '', sources: '' }
      messages.value.push(assistantMsg)

      try {
         const response = await fetch(
                 `http://localhost:8080/api/chat?
query=${encodeURIComponent(query)}`
         )
         const reader = response.body!.getReader()
         const decoder = new TextDecoder()

         while (true) {
            const { done, value } = await reader.read()
            if (done) break
            const chunk = decoder.decode(value, { stream: true })

            // 检测来源信息
            if (chunk.includes('📚 来源:')) {
               const parts = chunk.split('📚 来源:')
               assistantMsg.content += parts[0]
               assistantMsg.sources = parts[1]
            } else {
               assistantMsg.content += chunk
            }
            scrollToBottom()
         }
      } catch (error) {
         assistantMsg.content += `\n[系统错误：${error}]`
      } finally {
         loading.value = false
      }
   }
</script>

<style scoped>
   .chat-container {
      max-width: 800px;
      margin: 0 auto;
      padding: 20px;
      height: 100vh;
      display: flex;
      flex-direction: column;
   }

   .messages {
```

```
    flex: 1;
    overflow-y: auto;
    padding: 10px;
}

.message {
    margin-bottom: 15px;
    padding: 15px;
    border-radius: 8px;
}

.user {
    background-color: #e3f2fd;
    text-align: right;
}

.assistant {
    background-color: #f5f5f5;
}

.role {
    font-weight: bold;
    margin-bottom: 5px;
    font-size: 0.9em;
    color: #666;
}

.content {
    line-height: 1.6;
}

.sources {
    margin-top: 10px;
    padding-top: 10px;
    border-top: 1px solid #ddd;
    font-size: 0.85em;
    color: #666;
}

.input-area {
    display: flex;
    gap: 10px;
    padding: 15px 0;
}

.input-area input {
    flex: 1;
    padding: 12px;
    border: 1px solid #ddd;
    border-radius: 6px;
    font-size: 14px;
}

.input-area button {
```

```
        padding: 12px 24px;
        background-color: #1890ff;
        color: white;
        border: none;
        border-radius: 6px;
        cursor: pointer;
    }

    .input-area button:disabled {
        background-color: #ccc;
        cursor: not-allowed;
    }

    .loading {
        text-align: center;
        color: #999;
        padding: 10px;
    }
</style>
```

附录 D：故障排查决策树

```
系统无法启动
    │
    ├─ Docker 容器启动失败
    │   ├─ 检查 docker logs <容器名>
    │   ├─ 检查 docker stats 内存占用
    │   └─ 检查 daemon.json 格式
    │
    ├─ Java 应用启动失败
    │   ├─ 检查 JVM 参数 (-Xmx)
    │   ├─ 检查 application.yml 配置
    │   └─ 检查 Maven 依赖是否完整
    │
    └─ Ollama 服务不可用
        ├─ 检查 systemctl status ollama
        ├─ 检查 OLLAMA_HOST 配置
        └─ 检查安全组端口 11434

检索失败
    │
    ├─ Connection Refused 19530
    │   ├─ 检查 Milvus 容器状态
    │   ├─ 检查安全组端口 19530
    │   └─ 检查 host 配置 (服务名 vs localhost)
    │
    ├─ 维度不匹配错误
    │   ├─ 检查 embedding-dimension 配置
    │   ├─ 检查模型实际维度
    │   └─ 删除旧 Collection 重建
```

```
    │
    └─ collection not found
        ├─ 检查 VectorStoreConfig.initializeSchema(true)
        ├─ 检查 collection-name 配置
        └─ 手动创建 Collection

响应缓慢
    │
    ├─ 首字延迟 > 5s
    │   ├─ 检查 Ollama 模型是否预热
    │   ├─ 检查 Swap 使用率
    │   └─ 考虑降级模型 (7b→1.5b)
    │
    ├─ 检索延迟 > 500ms
    │   ├─ 检查 Milvus 索引类型
    │   ├─ 检查向量维度匹配
    │   └─ 检查网络延迟
    │
    └─ 内存持续高涨
        ├─ 检查 OLLAMA_KEEP_ALIVE 配置
        ├─ 检查 Docker 内存限制
        └─ 增加 Swap 空间
```

附录 E：2026 年技术趋势与展望

E.1 RAG 技术演进方向和路线

阶段	时间	特征	代表技术
Naive RAG	2023	基础检索 + 生成	简单向量检索
Advanced RAG	2024	检索优化 + 后处理	Rerank、查询重写
Modular RAG	2025	模块化、可组合	混合检索、GraphRAG
Agentic RAG	2026	智能体自主决策	Self-RAG、CRAG

趋势	说明	影响	时间线
多模态 RAG	支持图片、音频、视频的检索与生成	应用场景大幅扩展	2026-2027
Agent 框架	智能体自主规划与工具调用	系统复杂度提升	2026
边缘部署	模型压缩与端侧推理	降低云端依赖	2026-2028
向量数据库演进	混合检索、图数据库融合	检索精度持续提升	持续演进

E.2 2026 年值得关注的技术和推荐关注项目

技术	描述	成熟度	建议
GraphRAG	结合知识图谱的 RAG	◍ 发展中	关注，适合复杂关系场景

技术	描述	成熟度	建议
Self-RAG	模型自我评估检索质量	🟡 发展中	实验性采用
多模态 RAG	支持图像、音频检索	🟢 可用	生产环境可考虑
RAG + Agent	RAG 与智能体结合	🟢 可用	2026 年主流趋势
Tiny Embedding	超小型嵌入模型 (<100M)	🟢 可用	低配服务器首选

Spring AI 1.x： 持续跟进 GA 版本更新
Milvus 2.6+： 关注新索引类型与性能优化
Ollama： 关注新模型支持与量化优化
LangChain4j： Java 生态的 RAG 框架 alternative

E.3 成本优化建议

优化方向	具体措施	预期节省
模型选择	使用量化模型 (Q4_K_M)	内存 -60%
缓存策略	Redis 缓存热点查询	API 调用 -70%
索引优化	DiskANN 替代 HNSW	内存 -50%
云端 API	混合使用本地 + 云端	成本 -40%
自动扩缩容	根据负载动态调整	资源 -30%

附录 F：完整检查清单（2026 版）

F.1 部署前检查清单

- ☐ **硬件资源确认**
  - ☐ CPU: ≥2 核（推荐 4 核+）
  - ☐ 内存：≥4GB（推荐 8GB+）
  - ☐ 硬盘：≥50GB SSD
  - ☐ 网络：公网 IP + 安全组配置
- ☐ **软件环境确认**
  - ☐ Docker 20.10+ 已安装
  - ☐ Docker Compose v2+ 已安装
  - ☐ JDK 21 已安装
  - ☐ Ollama 已安装并测试
- ☐ **配置确认**
  - ☐ application.yml 配置完整
  - ☐ docker-compose.yml 内存限制已设置
  - ☐ Swap 分区已创建（低配服务器）
  - ☐ 安全组端口已开放

- ☐ **模型确认**
  - ☐ Embedding 模型已拉取 (bge-m3)
  - ☐ Chat 模型已拉取 (qwen2.5:1.5b)
  - ☐ 模型预热已完成
- ☐ **安全确认**
  - ☐ 敏感端口已限制访问 IP
  - ☐ 数据库密码已修改
  - ☐ 日志脱敏已配置

F.2 组件部署检查

- ☐ Milvus 容器状态 healthy
- ☐ Milvus 内存占用 < 1GB
- ☐ Ollama 模型已拉取并预热
- ☐ Ollama 内存限制已配置
- ☐ Spring Boot JVM 参数已限制

F.3 功能验证检查

- ☐ 文档上传成功
- ☐ 向量检索正常
- ☐ 流式响应正常
- ☐ 引用来源展示正常
- ☐ 内存监控正常

F.4 生产环境检查

- ☐ HTTPS 已配置
- ☐ 用户认证已实现
- ☐ 日志审计已开启
- ☐ 数据备份已配置
- ☐ 监控告警已部署

F.5 开源代码仓

https://github.com/SuniaW/ai-learn.git

结语

恭喜您完成了企业级 RAG 智能问答系统的全栈实施！

本指南涵盖了从基础概念到生产部署的完整流程，特别针对低配环境进行了深度优化。记住以下核心原则：

原则	说明	重要性
资源精细化管理	在有限内存下，每个组件都要划定 "红线"	必须

原则	说明	重要性
模型选型务实	不追求参数大，追求系统链路通畅	🔴 必须
监控先行	部署后立即建立监控体系，防患于未然	🔴 必须
持续优化	根据实际运行情况不断调整参数配置	🟡 推荐

本书涵盖的技术栈：

- ✅ Spring AI 1.0 GA（2025 年 5 月正式发布）
- ✅ Milvus 2.6（2025 年 6 月发布，内存减少 72%）
- ✅ Ollama 最新优化技巧（2026 年实践）
- ✅ Embedding 模型 MMTEB 评估基准（ICLR 2025）
- ✅ 企业级 RAG 安全与合规（2026 年标准）
- ✅ 最新内存隐形占用排查与清理实战

从入门到生产的完整路径：

```
学习阶段 → 原型开发 → 性能优化 → 生产部署 → 运维监控
   ↓          ↓          ↓          ↓          ↓
 概念理解    Spring AI   Milvus 调优  容器化     Prometheus
 RAG 原理    Milvus Lite  Ollama 优化  安全加固   告警体系
```

最后提醒：

1. **版本选择：** 生产环境请使用 Spring AI 1.0.2+ 和 Milvus 2.6+
2. **模型选型：** 参考 MMTEB 排名，不要盲目追求大模型
3. **安全第一：** 企业级部署必须做好数据脱敏和访问控制
4. **持续优化：** RAG 系统需要持续评估和迭代优化
5. **内存警惕：** 时刻关注"隐形"内存占用，定期执行 Docker 和系统级清理（参考 28.4 节）

祝您的 RAG 系统在 2026 年运行顺利！ 🚀